

Confide - Anonymous Academic Chat System

Bruno Tavares, n.º 49217

Margarida Toureiro, n.º 49519

Ricardo Almeida, n.º 46927

Supervisor: Pedro Félix

Final Project Report for the course *Project and Seminar*
Bachelor's Degree in Computer Science and Computer Engineering
July 2025

Acknowledgements

We would like to express our gratitude to all those who contributed to the success of this project especially our supervisor, Professor Pedro Félix, for his guidance and support throughout the development of the System. His expertise and insights were instrumental in shaping the direction of our work. We would also like to thank our families, friends and colleagues for their encouragement and support during the development of the System and for their patience during the long hours of work. Their support and understanding were invaluable to us. Thank you all for your contributions and support, which made this project possible.

Abstract

This report presents the design and implementation of an Anonymous Academic Chat System developed as part of the course *Project and Seminar*. Anonymous platforms have been shown to foster more open and inclusive discussions by reducing participants' fear of judgment. Nevertheless, such platforms also carry a significant risk of misuse since user identities are not disclosed. The proposed solution aims to facilitate anonymous communication among students and teachers while maintaining a mechanism for accountability, thus encouraging an active and responsible involvement in academic settings. The system provides a way for institutions to create their own instances, allowing the configuration of a group of valid email domains and to organize the categories in a tree-like structure, making it possible to emulate the institution's structure and allowing users to navigate through the hierarchy. Permissions within the system are fully customizable by the teachers, meaning that they can enforce anonymity within a specific Category or allow users to interact using their real identities. Anonymity is achieved through the use of a ticket system, where users are given tickets to create and prove that they are the owners of a specific pseudonym. The ticket that proves the ownership is not stored on the server, meaning that the system never persists any data that associates a specific user to a pseudonym. Accountability is ensured by using Asymmetrical Encryption, the system persists a link between a Pseudonym and a User's ciphered identification. The key that enables deciphering this identification is a shared secret between the system administrators, meaning that unless two or more administrators participate in the unveiling action, the key cannot be obtained, thus preserving the user's anonymity.

Resumo

Este relatório apresenta o desenho e implementação de um Sistema de Chat Académico Anónimo, desenvolvido no âmbito da unidade curricular **Projeto e Seminário**. Estudos indicam que plataformas anónimas promovem discussões mais abertas e inclusivas, ao minimizarem o receio de ser julgado por parte dos seus participantes. No entanto, tais plataformas também apresentam um risco significativo de uso indevido, uma vez que as identidades dos utilizadores não são divulgadas. A solução proposta tem como objetivo facilitar a comunicação anónima entre estudantes e docentes, mantendo ao mesmo tempo um mecanismo de responsabilização, estimulando um envolvimento ativo e responsável em ambientes académicos. O Sistema permite que as instituições criem as suas próprias instâncias, permitindo a configuração de um grupo de domínios de e-mail válidos e organização numa estrutura semelhante a uma árvore, tornando possível emular a estrutura da instituição e permitindo que os utilizadores naveguem através da hierarquia definida. As permissões do Sistema são personalizáveis pelos professores, o que permite a imposição do anonimato dentro de uma categoria específica ou permitir que os utilizadores interajam usando as suas identidades reais. O anonimato é alcançado através de um sistema de bilhetes, onde os utilizadores recebem bilhetes para criar e comprovar que são os proprietários de um pseudónimo específico. O bilhete que comprova a propriedade não é armazenado no servidor, o que significa que o Sistema nunca persiste quaisquer dados que associem um utilizador específico a um pseudónimo. A responsabilização é garantida através do uso de criptografia assimétrica, onde o Sistema persiste uma ligação entre um pseudónimo e uma identificação cifrada do utilizador. A chave que permite decifrar esta identificação é um segredo partilhado entre os administradores do Sistema, o que significa que, a menos que dois ou mais administradores participem na ação de revelação, o segredo não pode ser obtido, preservando assim o anonimato do utilizador.

Contents

1	Introduction	1
1.1	Context	1
1.1.1	Educational Institution structure	1
1.2	Solution	2
1.2.1	Usage in Academic context	3
1.2.2	System Permission Management	3
1.2.3	Typical use case	4
1.3	Other Solutions	5
1.3.1	Existing Solutions	5
1.3.2	Summary	7
1.4	Report Organization	7
2	System requirements	9
2.1	Functional Requirements	9
2.2	Non-functional Requirements	10
3	Architecture	11
3.1	System Components	11
3.2	Technology Stack	12
3.3	Frontend component	12
3.4	Backend component	13
3.5	Component Interaction	13
4	Data Model	15

4.1	Entity-Relationship diagram	15
4.1.1	Relationship Overview	15
4.2	Physical Data Model	19
4.2.1	Content DB	23
4.2.2	Sensitive DB	23
5	Frontend	25
5.1	Technology	25
5.1.1	Research	26
5.1.2	Libraries	29
5.2	Organization	31
5.3	Navigation	35
5.4	Networking	35
5.5	Platform Specific Code	36
5.6	Testing	37
5.6.1	Unit Testing	38
5.6.2	Widget Testing	38
5.6.3	Integration Testing	38
5.6.4	Differences	39
6	Backend	41
6.1	Load Balancer	41
6.2	Backend Services	42
6.2.1	Technology	42
6.2.2	Organization	42
6.2.3	HTTP API Endpoints	44
6.2.4	Authentication	44
6.2.5	Permissions	44
6.2.6	Server Sent Events	45
6.2.7	Database Access	46
6.2.8	Anonymity	48

6.3	Databases	49
7	Infrastructure	51
7.1	Overview	51
7.2	Cloud	52
7.3	Local	52
7.3.1	Machines	52
7.3.2	Automated configuration	52
7.4	GitHub	52
8	Conclusion	55
8.1	Achievements	55
8.2	Improvements and Future Work	56
8.3	Challenges Faced	57
8.3.1	Non-technical Challenges	57
8.3.2	Technical Challenges	58
8.4	Lessons Learned	58
8.4.1	Importance of Prototyping	59
8.4.2	Agile Approach	59
8.4.3	Collaboration and Communication	59
8.4.4	Documentation	59
9	Appendix	61
9.1	Detailed System entities	61
9.1.1	Content Database Entities	61
9.1.2	Sensitive Database Entities	64
9.2	API Endpoints	65

List of Figures

1.1	Typical structure of an educational institution	2
1.2	Typical use case of the System	5
3.1	System Architecture	11
4.1	Entities related to the system state	16
4.2	Entities directly related to user authentication	16
4.3	Entities directly related to groups and permissions	17
4.4	Entities directly related to chats and nodes	18
4.5	Entities directly related to messages	19
4.6	Tables directly related to user authentication	20
4.7	Tables directly related to groups and permissions	21
4.8	Tables directly related to chats and nodes	21
4.9	Tables directly related to messages	22
4.10	Tables directly related to messages	22
4.11	Physical Data Model of the Sensitive DB	24
5.1	Frontend Interactions	25
5.2	Example of the tree with Provider	29
5.3	Example of ChangeNotifier for recomposition	30
5.4	Dependency management in Main Application	32
5.5	Router access to dependencies	32
5.6	Content zone of a Screen	33
5.7	Content zone of a View	33
5.8	Content zone of multiple Widgets of the same type	34

5.9	The structure of a common Repository	34
5.10	Navigation overview	35
5.11	InterceptorClient use	36
6.1	Backend Architecture	41
6.2	Backend Service Architecture	42
6.3	Backend Service Organization	43
7.1	Infrastructure overview	51

List of Tables

5.1	Summary of Flutter as a Development Technology	27
5.2	Summary of React Native as a Development Technology	28
5.3	Summary of Compose Multiplatform as a Development Technology	28
5.4	Comparison of Flutter Test Types	40

Listings

5.1	Class extends ChangeNotifier	30
5.2	Widget observes FavoriteChatModel	31
5.3	Media Query	36
5.4	Drawer creation based on Media Query result	36
5.5	Maestro YAML script	38
5.6	Use of label from Flutter's Semantics widget	39
6.1	Transaction Manager Signature	46
6.2	Data Transaction Interface Signature	47
6.3	Data Transaction Manager	47
6.4	Secure Transaction Interface Signature	47
6.5	Secure Transaction Manager	48

Chapter 1

Introduction

This report outlines the design and implementation of an Anonymous Academic Chat System, developed as part of the course *Project and Seminar*. The project was undertaken by a team of students, with the goal of creating a platform that allows for anonymous communication while maintaining accountability in academic discussions.

1.1 Context

In educational settings, effective communication is important for successful learning, especially when it comes to student engagement in academic discussions. Research has shown that these discussions promote the understanding of material and facilitate the learning process of individuals and the class as a whole [1], [16], [22]. Moreover, these discussions are frequently a good indicator of student motivation [15].

Yet, encouraging students to actively participate in said discussions is an ongoing challenge. Students hesitate to ask questions due to shyness or concerns about criticism or potential repercussions. Research indicates that students often struggle with communication due to fear and anxiety, particularly in oral interactions [12]. Additionally, these challenges appear to be more pronounced among female students, who tend to experience higher levels of anxiety in both oral and written communication compared to their male counterparts [12].

1.1.1 Educational Institution structure

An educational institution is typically structured into several levels, where each level can contain departments, services, and other organizational units. Focusing on the academic organization, an institution is usually divided into different departments, each focused on a specific field of study. The departments contain specific degree programs, which are further divided into courses. Each course is taught by one or more teachers, who are responsible for

delivering the course content and assessing student performance. Each course then has one or more classes, containing students enrolled in that course. Figure 1.1 illustrates a typical structure of an educational institution, focusing on the academic departments.

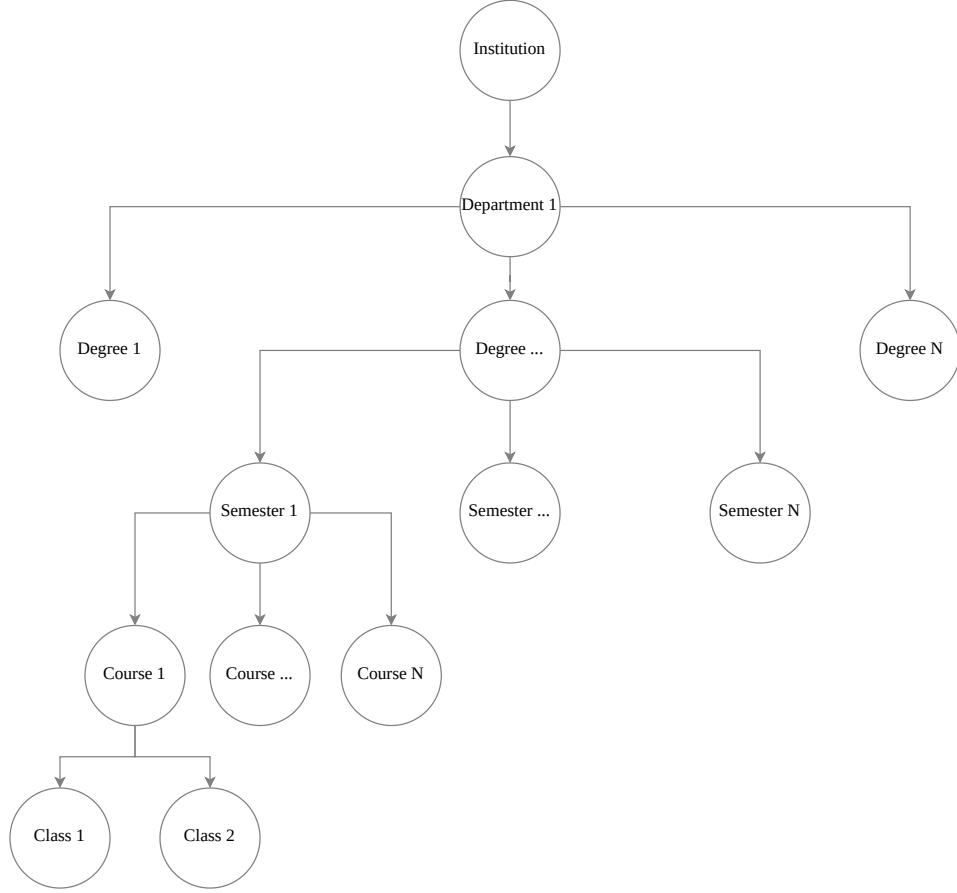


Figure 1.1: Typical structure of an educational institution

1.2 Solution

Anonymous chats are unique spaces where strangers connect freely, authentically, and without fear of judgment. The anonymity factor encourages more open discussions, making room for individuals to comfortably expose their questions and concerns [2]. By implementing a communication platform that protects student identities, academic institutions can create a more inclusive and engaging learning environment, as anonymity has been shown to increase participation in student discussion forums [20].

Hence, the proposed solution is a multi-chat communication platform for students and teachers, capable of ensuring that student identities remain undisclosed, even across different chats. This solution promotes active participation while fostering a supportive atmosphere for both

students and educators.

However, being anonymous also has downsides. Students may use that freedom irresponsibly, engaging in harmful behaviours such as harassment and chat disruption [17]. Without accountability, a student might feel empowered to act in ways they would not if their real identity were known [18].

To mitigate the downside of anonymity, the system was designed so that only students registered with their institutional email can join, thus linking their real identity to any potential abusive behavior, which can be revealed under the explicit action of at least two administrators.

For consistency, this solution will hereby be referred to as System throughout this report.

1.2.1 Usage in Academic context

As described in section 1.1.1, educational institutions are typically structured into several levels, with each level containing departments, services, and other organizational units. The System is designed to support different possible structures of educational institutions, allowing for flexibility in its implementation. The System provides a tree-like structure, meaning that the Administrators can configure the system to match the specific structure of their institution. This structure can be used to create different chat rooms for each department, course, or class, allowing students and teachers to communicate within their specific context. The System can be configured to represent the institution's structure or, alternatively, to represent only the structure of a specific course or class. This flexibility allows the System to be adapted to different educational contexts.

1.2.2 System Permission Management

The System implements a permission management system that allows administrators to control access to different features and functionalities. The System allows for the creation of different roles with specific permissions, which can be assigned to users based on their needs and responsibilities. Below are some of the permissions defined in the System:

- **Create Nodes:** Users with this role can create new Nodes within the system, representing different levels of the educational institution's structure.
- **Create chats:** Users with this role can create new chats within the system, for a specific node.
- **See Nodes:** Users with this permission can view specific Nodes within the system, allowing them to navigate through the educational institution's structure.

- **Be Pseudonym:** Users with this permission are assigned a pseudonym, which is used to represent them in the system. This pseudonym is unique and allows users to communicate anonymously.

1.2.3 Typical use case

The System can be used in various scenarios, depending on the needs of the educational institution. A possible use case at the start of a semester is as follows:

1. A teacher creates a Group with the typical student permissions, such as seeing nodes and being pseudonym.
2. The teacher creates a Node for current semester.
3. The teacher creates Nodes for each Course offered in the semester, inside the semester node.
4. The teacher creates Nodes for the different classes, inside each Course node.
5. The teacher creates Chats as needed, inside each class Node, allowing students to view the chat.
6. The teacher assigns the Group to the Nodes.
7. The teacher adds students to the Group, allowing them to access the Node and its chats.
8. Students can join the chat and communicate anonymously with each other and the teacher.

Figure 1.2 illustrates the organizational tree of the described scenario.

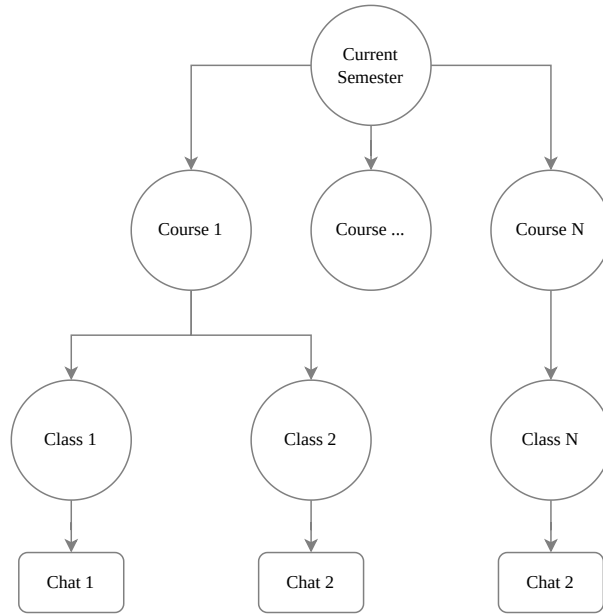


Figure 1.2: Typical use case of the System

1.3 Other Solutions

The solution presented in this report aims to be an Anonymous Academic Chat, striking a balance between anonymity and accountability. It is novel in that it combines registered participation (using users’ institutional email addresses), with unlinkable pseudonyms per chat (preventing correlation of pseudonyms across chats), and a threshold-based cryptographic deanonymization mechanism (using Shamir’s Secret Sharing, [21]).

In our research, several existing solutions were found that support anonymous or privacy-preserving communication. However, none of them implement all three core features listed above.

1.3.1 Existing Solutions

SimpleX

SimpleX [14] is a messaging application designed for privacy and anonymity. It avoids using user accounts or servers that store messages, making communication harder to trace. Conversations are based on temporary identifiers that cannot be linked to each other. While it supports per-chat anonymity, there is no user registration or way to reveal someone’s identity, even if needed.

Summary: Anonymity and unlinkable chats, but lacks user registration and any way to identify users if needed.

Signal

Signal [6] is a secure messaging app known for its encryption and privacy tools. It requires a phone number to register and users keep the same identity across all chats. It doesn't support unlinkable pseudonyms or any way to reveal who someone is in case of misconduct.

Summary: Strong encryption and privacy, but no per-chat pseudonyms or identity reveal option.

Matrix

Matrix [7] is a decentralized chat system that supports encrypted communication and user accounts. While users can choose a display name, their identity remains the same across chats and can be seen by the server administrators. It does not provide unlinkable pseudonyms or cryptographic identity reveal.

Summary: Registered users and encrypted chats, but pseudonyms are linked and admins can access identity information.

Session

Session [5] focuses on anonymous messaging using a randomly generated ID instead of registration. It offers encrypted chats and stores conversation history. However, like SimpleX, there's no way to connect a Session ID to a real user or reveal identity in case of issues.

Summary: Offers anonymous, persistent chats, but lacks user registration and any way to trace identities.

Threema

Threema [9] is a messaging app that lets users stay pseudonymous through randomly generated IDs. It offers message history and encryption, but users keep the same ID across chats and there's no way to reveal their identity without their consent.

Summary: Supports secure messaging with pseudonyms, but pseudonyms are reused across chats and not unlinkable.

1.3.2 Summary

The existing solutions provide various levels of anonymity and security, but none seemed to fill the specific niche and use of the solution presented in this report. The combination of registered participation, unlinkable pseudonyms per chat, and a threshold-based deanonymization mechanism is unique and addresses the need for both anonymity and accountability in academic discussions.

1.4 Report Organization

This report is organized into several chapters, each focusing on different aspects of the System's design and implementation. The chapters are structured as follows:

- **Introduction 1** - Provides an overview of the project, its objectives, context in which it was developed and enumerates some similar solutions.
- **System Requirements 2** - Details the functional and non-functional requirements of the System, outlining what the System must achieve.
- **Architecture 3** - Describes the overall architecture of the System, including the Frontend and Backend components, their interactions, and naming conventions.
- **Data Model 4** - Presents the data model used in the System, including the structure of the database and how data is organized.
- **Frontend 5** - Discusses the Frontend component of the System, including the technologies used, design choices, and implementation details.
- **Backend 6** - Covers the Backend component of the System, detailing the architecture, services, and technologies used to implement the server-side logic.
- **Infrastructure 7** - Explains the infrastructure of the System, including Continuous Integration (CI) and Continuous Deployment (CD) processes, the local development environment, and the live production environment.
- **Conclusion 8** - Summarizes the project, discusses the outcomes, and reflects on the development process.
- **Appendix 9** - Contains additional information and other relevant details that support the main content of the report.

Chapter 2

System requirements

This chapter outlines the requirements for the System, detailing both functional and non-functional aspects.

2.1 Functional Requirements

The System functional requirements are enumerated below, focusing on the core features and functionalities that the System must support:

- The System must provide an onboarding process, allowing initial configuration of the system (administrators, root node and allowed domains).
- The System has a node-tree organization, configurable by the academic institution.
- The System allows customizable user permission configurations.
- The System allows users to create nodes.
- The System allows users to create chats within nodes.
- The System provides a real-time chat experience, allowing users to send and receive messages instantly.
- The System allows management of messages (validation/removal).
- Users must register in the System using their institutional email (valid domain).
- Users must confirm ownership of their institutional email by clicking a link sent to that email address.
- Users can be anonymous in the System, interacting through pseudonyms.
- Users can search for chats.

- Users can send messages in chats they can see.
- Users can favorite chats.
- Users see an unread message count for each favorite chat.
- Users can report another user's message for misconduct or violating the rules in a given chat.
- The identity of a user engaging in misconduct can be revealed through the deliberate action of multiple System administrators.
- In the event of a database breach, users' anonymity will still be preserved.
- The System must ensure that a user only has a single pseudonym per chat, ensuring accountability.

2.2 Non-functional Requirements

The non-functional requirements are listed below, focusing on the quality attributes that the System must exhibit:

- **Performance:** The System must deliver low-latency, real-time communication with minimal delay, even with many concurrent users, ensuring a smooth user experience.
- **Availability:** The System should be highly available, ensuring minimal downtime and reliable access, with mechanisms in place for automatic recovery from failures.
- **Scalability:** The System should support horizontal scaling, where additional servers or containers can be added dynamically to meet demand.
- **Portability:** The System should be accessible on both Web and Mobile, providing a consistent user experience across devices and screen sizes.
- **Testing:** Emphasis on unit and integration tests to ensure strong and reliable results.

Chapter 3

Architecture

This chapter represents the architecture of the System, its main components and how they interact with each other. Adding to this, it will also set the standard for terms that will be used through the rest of the document. The System is composed of two main components: The Frontend and the Backend. These will be explained in detail in chapters 5 and 6, respectively. The Frontend and the Backend components are illustrated in figure 3.1.

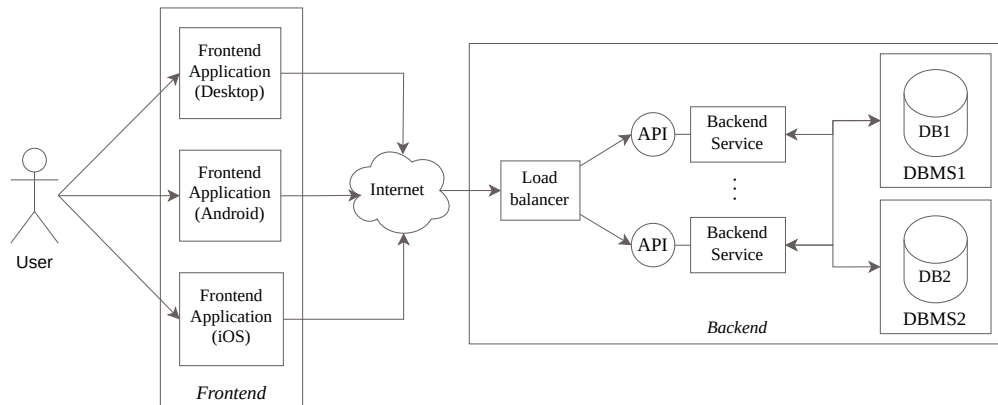


Figure 3.1: System Architecture

3.1 System Components

The following terms will be used throughout the report to ensure clarity and consistency:

- **System** - Refers to the entire application being developed, which includes both the Frontend and Backend components.
- **Frontend** - Refers to the user interface and client-side components of the System. The Frontend consists of the Frontend Applications. The Frontend is explained in 5.

- **Frontend Applications** - Refers to the specific applications that make up the Frontend. Frontend Applications are explained in 5.
- **Backend** - Refers to the server-side components, including business logic and data management. The Backend consists of the Load Balancer, the Backend Services and the Database Management Systems (DBMS). The Backend is explained in 6.
- **Load Balancer** - Refers to the component that distributes incoming requests across multiple Backend Services to ensure high availability and reliability. The Load Balancer is explained in 6.1.
- **API** - Refers to the Restful HTTP Application Programming Interface that allows the Frontend Applications to communicate with the Backend Services. The API is explained in 6.2.3.
- **Backend Services** - Refers to the individual services that handle specific business logic and data processing tasks within the Backend. The Backend Services are explained in 6.2.
- **DBMS** - Refers to the Database Management Systems used by the Backend. The managed databases are explained in 6.3.

The Frontend and Backend components will be further detailed in their respective chapters, where the architecture, technologies, and design choices will be discussed in depth.

3.2 Technology Stack

Based on the requirements of the project and team experience, the following technology stack was chosen for the System:

- **Frontend Applications:** The Frontend Applications were developed using *Flutter* [10].
- **Load Balancer:** The Load Balancer was implemented using *Nginx* [4].
- **Backend Services:** The Backend Services are Kotlin applications, using the *Spring Boot* framework [19], that run on the Java Virtual Machine (JVM)
- **DBMS:** The databases are managed using *PostgreSQL* [11].

3.3 Frontend component

The Frontend is the user-facing part of the System, responsible for providing an intuitive and responsive interface for users to interact with. One of the objectives the team set for

the Frontend, other than to provide a good user experience, was to maximize the amount of code that is reused across the different Frontend Applications. In order to achieve this, the team chose to use a multi-platform framework. After evaluating several options, the team decided to use *Flutter* as the framework for the Frontend. The investigative work and decision-making will be detailed in the chapter 5 of this report.

3.4 Backend component

The Backend is the server-side part of the System, responsible for processing requests from the Frontend, managing data, and implementing business logic. The Backend is designed to be scalable and reliable. The Backend component includes a Load Balancer, multiple Backend Services, and two instances of database management systems. The parts of the Backend will be detailed in the chapter 6 of this report.

3.5 Component Interaction

The interaction between the Frontend and Backend components is what enables the overall functionality of the System. The Frontend Applications communicate with the Backend Services through a well-defined HTTP API. This API allows the Frontend to send requests to the Backend Services, which process the requests and return the appropriate responses. The Load Balancer ensures that requests are distributed evenly across the Backend Services, while the Database provides persistent storage for the data managed by the Backend. This architecture allows for efficient handling of user requests and ensures that the System can scale to accommodate a growing number of users.

Chapter 4

Data Model

This chapter describes the data model of the System. The data model defines how the data is structured and organized within the System, including the relationships between different entities.

4.1 Entity-Relationship diagram

To understand how the entities in our System relate to each other, the team started by designing an Entity-Relationship (ER) diagram.

4.1.1 Relationship Overview

To ensure clarity throughout this chapter, the team chose to present separate sections of the ER diagram grouped by shared logic, rather than displaying the complete diagram at once.

System State

Figure 4.1 shows the entities that represent the current state of the System. The `SystemState` entity contains information about the current state of the System, such as its version and configuration. The `SystemStates` contain the possible states of the System. The `SystemState` entity is linked to the `SystemStates` entity, allowing the System to transition between different states.

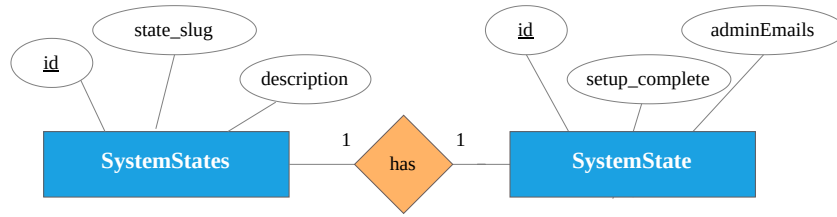


Figure 4.1: Entities related to the system state

User Authentication and Registration

Figure 4.2 illustrates the portion of the ER diagram focused on the User entity and associated authentication-related entities. The System supports email and password authentication, but the model allows for the possibility of adding other authentication methods in the future. Domains can be added or removed to control which email domains are allowed for user registration. The RegistrationToken ensures that users validate that they own the email address they registered with. The Authentication entity stores the encrypted password of the user, which is used to authenticate them when they log in.

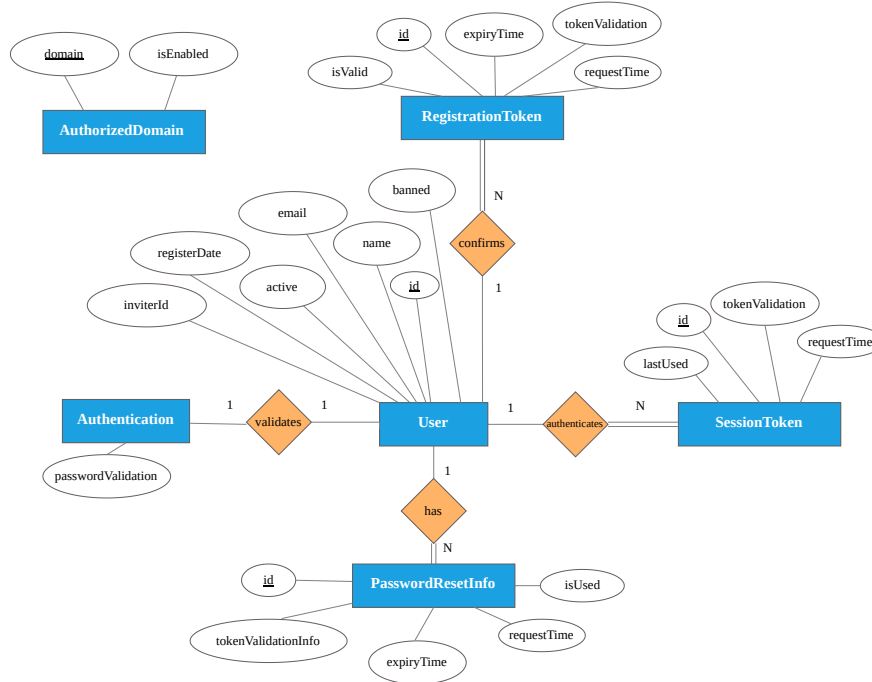


Figure 4.2: Entities directly related to user authentication

The main takeaway for the User Authentication and Registration section is the separation of concerns between user data and authentication data. The User entity contains general user information, while the Authentication entity is responsible for storing authentication-related

data. This separation gives the System flexibility to implement different authentication methods in the future.

Groups and Permissions

Figure 4.3 depicts the Group entity and its associated permission-related entities. The Group entity manages permissions within the System. The Permission entity defines the various permissions that can be assigned to groups. Permissions can be global or local, depending on whether they apply to the entire System or only to a specific node. The GroupChange entity stores logs of changes made to groups, allowing for auditing and tracking of modifications.

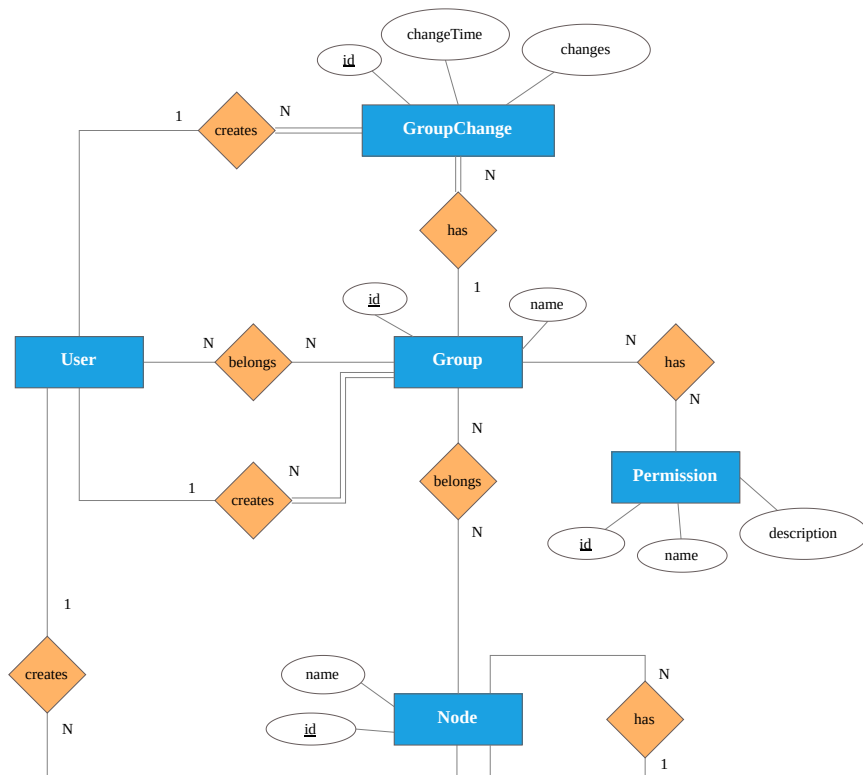


Figure 4.3: Entities directly related to groups and permissions

This design allows for many-to-many relationships between users and groups, enabling users to belong to multiple groups and groups to contain multiple users. A node can be assigned to multiple groups, and a group can be assigned to multiple nodes. This allows for flexible permission management across different parts of the System.

Chats and Nodes

Figure 4.4 illustrates the section of the diagram concerning Chat-related entities and their connection to the Node entity. The Node entity represents a category or topic within the System, which can be used to organize chats. The Chat entity represents a chat room where users can communicate. The UserFavorites entity allows users to mark chats as favorites, providing quick access to frequently used chats. Nodes are organized in a hierarchical tree structure, allowing for nested categories.

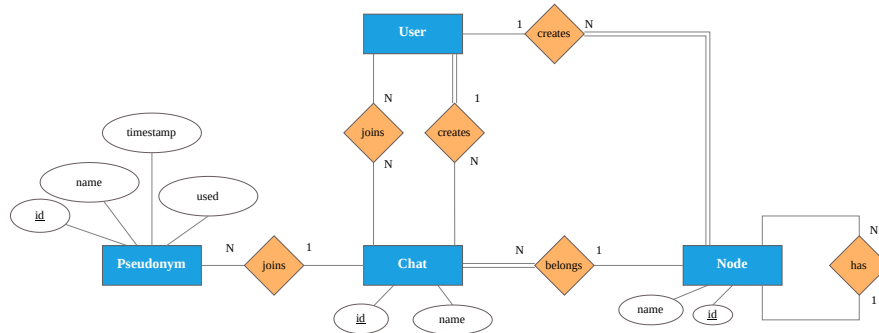


Figure 4.4: Entities directly related to chats and nodes

Note that Pseudonym and User entities are not directly connected in the ER diagram. This design choice is to ensure that pseudonyms are not directly linked to users in the database. A pseudonym is only associated with one chat, while a user can be associated with multiple chats. The Pseudonym entity represents the fake name that a user can use to communicate in a chat, allowing for anonymity.

Messages and Reporting

Figure 4.5 presents the entities related to the lifecycle of the Message entity, including message reporting and different message states. The Message entity represents a message sent within a chat. The MessageState entity defines the various states a message can be in. The MessageReport entity represents reports submitted for messages, allowing users to report inappropriate content.

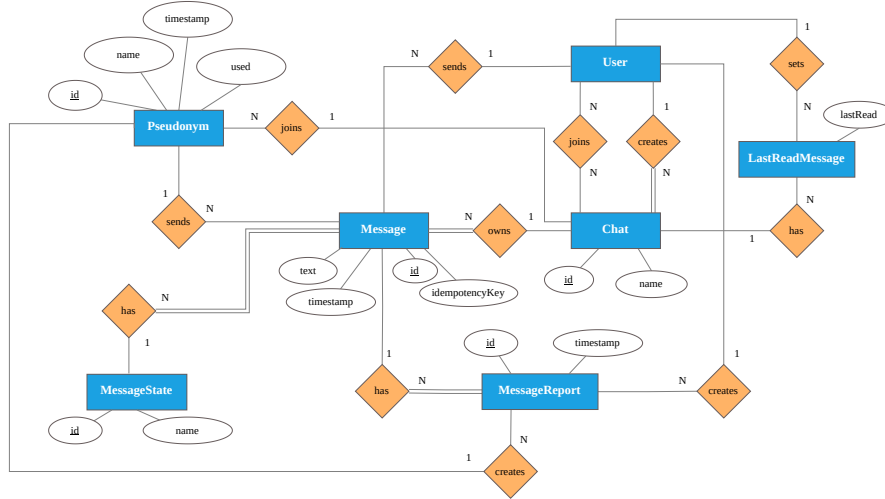


Figure 4.5: Entities directly related to messages

It is worth noting that a Message can be reported multiple times, and each report is stored in the MessageReport entity. This allows for multiple users to report the same message, which can be useful for moderation purposes. A report can be anonymous or associated with a specific user.

4.2 Physical Data Model

While the ER diagram offers a conceptual view of the System's data structure, the Physical Data Model translates it into a concrete relational database schema, with tables and restrictions.

Similarly to the previous diagram, the team has chosen to present separate sections of the Physical Model grouped by shared logic, for the sake of clarity.

Figure 4.6 illustrates the portion of the Physical Model focused on the `users` table and associated authentication-related tables, such as `authentications` and `registration_tokens`.

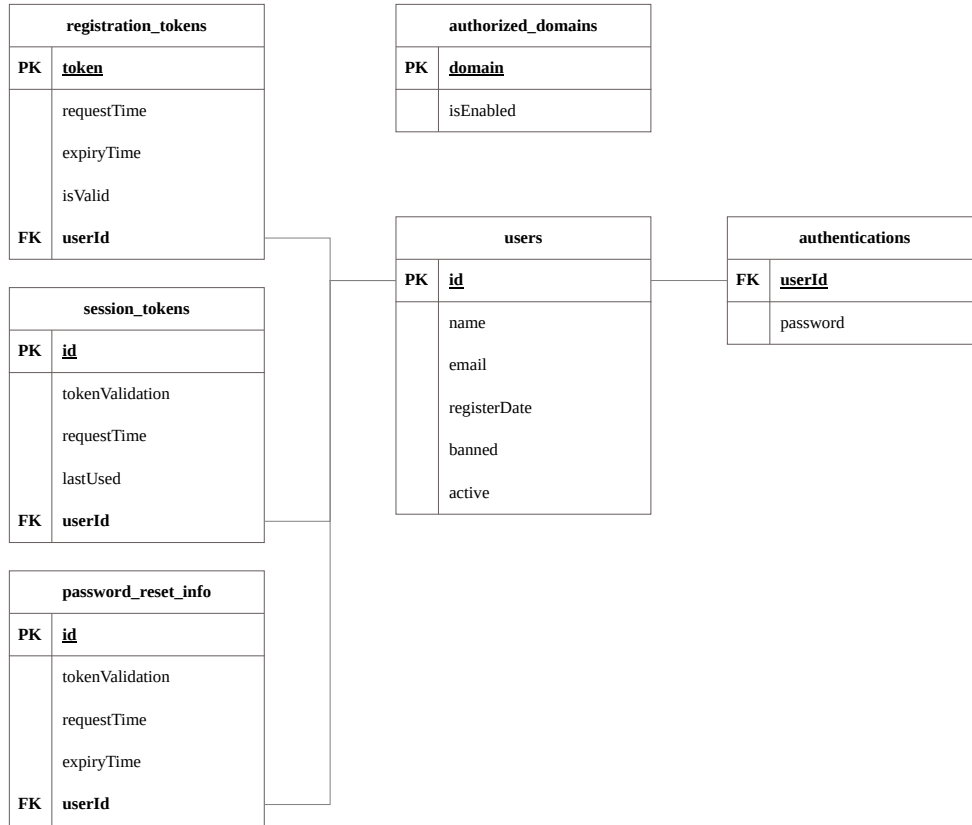


Figure 4.6: Tables directly related to user authentication

Figure 4.7 presents the tables related to **groups** and **permissions**. This section also introduces several junction tables, such as **user_groups**, **group_permissions** and **group_nodes**. The foreign key **parentId** in the **nodes** table references the id of another node. This association ensures a valid hierarchical tree structure within the System.

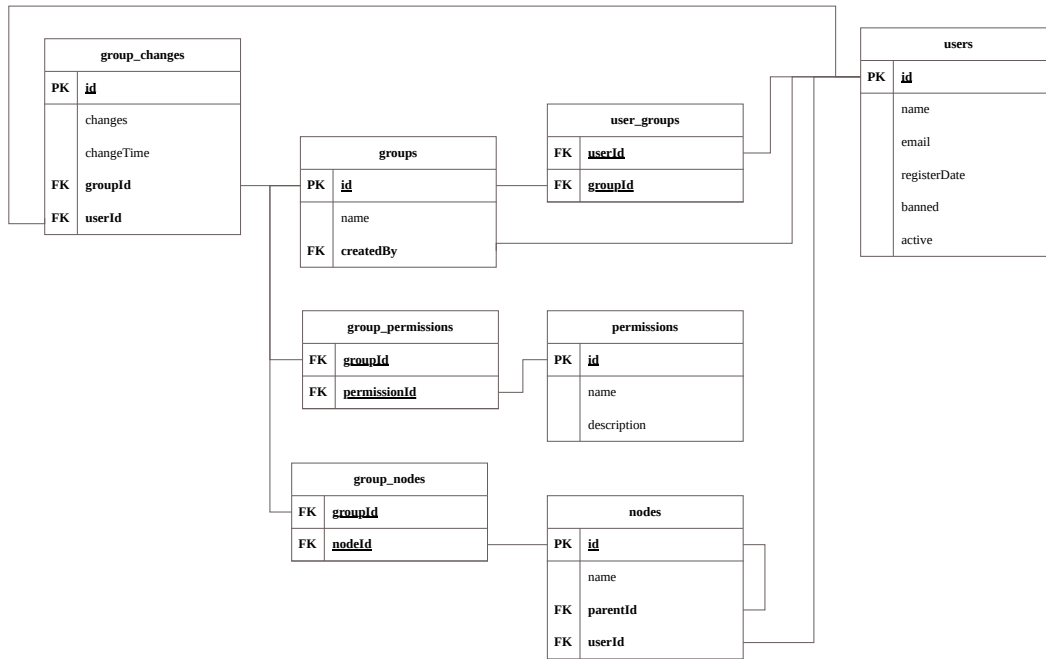


Figure 4.7: Tables directly related to groups and permissions

Figure 4.8 presents the part of the schema concerning **nodes** and **chats**. It includes the **user_chats** and **user_favorites** junction tables, which manage chat participation and the possibility for the users to mark a chat as favorite in the System.

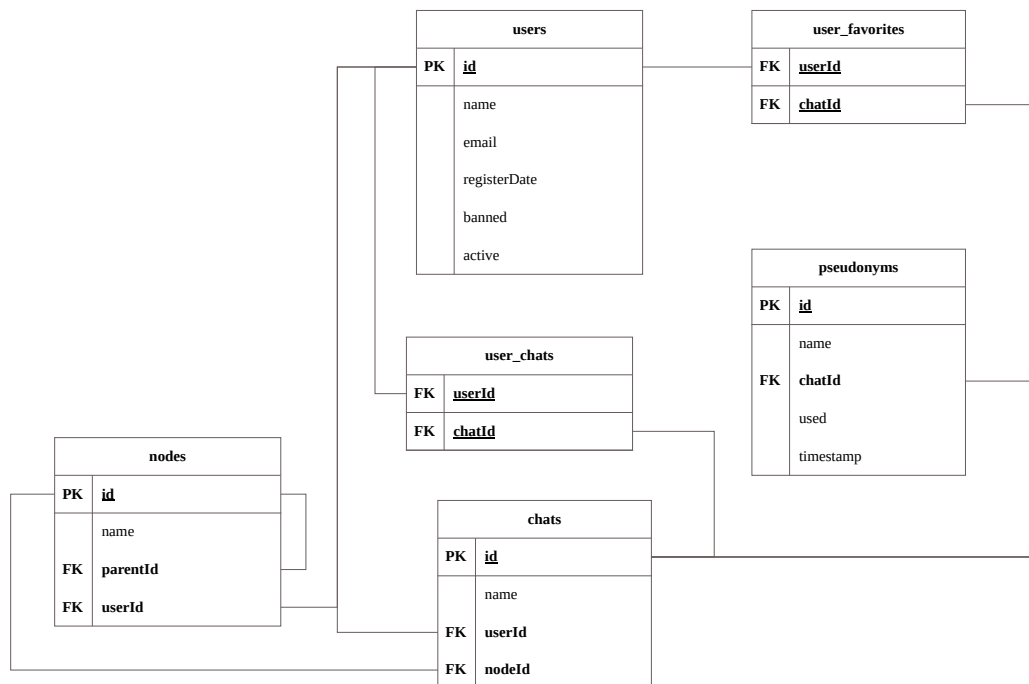


Figure 4.8: Tables directly related to chats and nodes

Figure 4.9 shows the portion of the schema dealing with `messages`, including message reporting and message states.

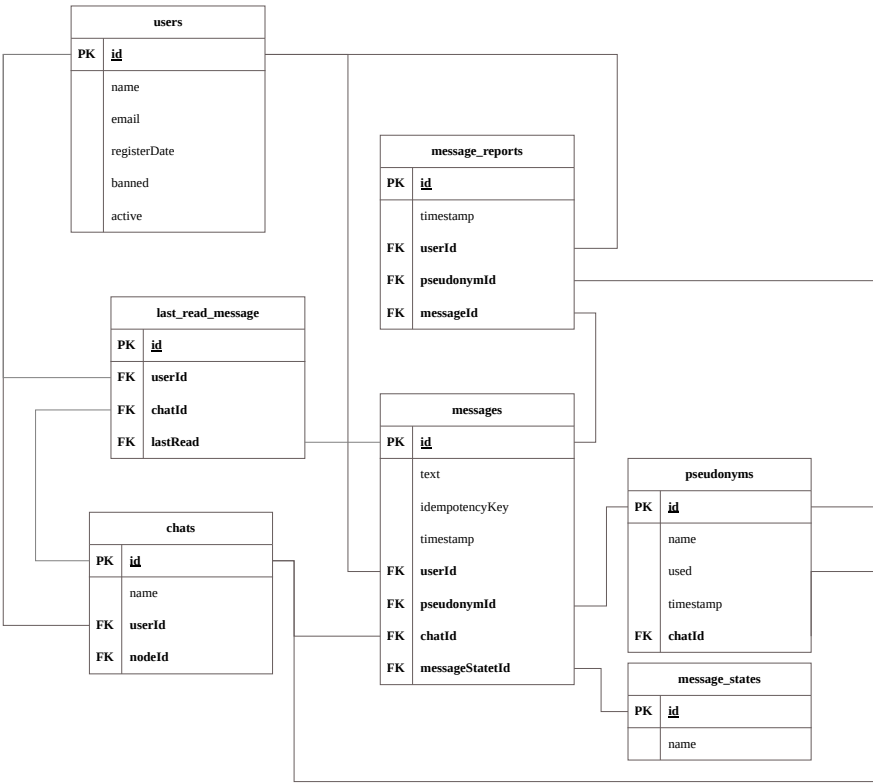


Figure 4.9: Tables directly related to messages

Figure 4.10 presents the tables that are directly related to the `SystemState` entity. The `system_states` table contains the possible states of the System, while the `system_state` table holds the current state of the System.

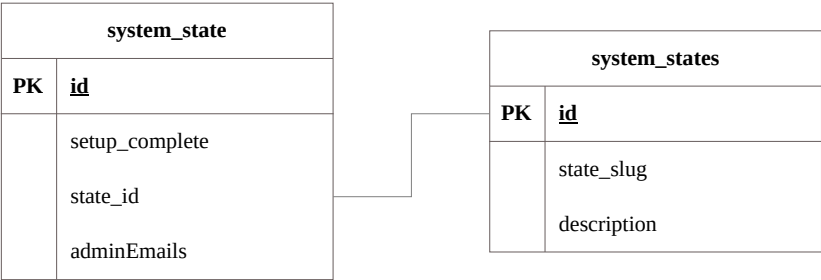


Figure 4.10: Tables directly related to messages

To support both core application functionality and specific requirements to ensure user privacy, the System implements a dual-database architecture. The two databases are:

- **Content DB** - the main operational database used by the System.
- **Sensitive DB** - a secondary database used to store sensitive mappings related to pseudonym ownership.

The databases are managed by separate Database Management Systems (DBMS), ensuring that the sensitive data is stored in a different physical location from the regular data. This separation enhances security by preventing direct access to sensitive information through the main operational database.

4.2.1 Content DB

The Content DB holds all data required for the core operations within the System to function. This database includes **users**, **chats**, **messages**, **groups**, **pseudonyms**, and other operational entities' tables. The fundamental aspect is that this database does not contain any associations between pseudonyms and real users. Pseudonyms are stored independently and are linked only to specific chats.

4.2.2 Sensitive DB

The Sensitive DB is a separate database used exclusively to store data that supports user anonymity and pseudonym ownership, without directly exposing user identities. This separation minimizes the potential for identity correlation between a user and their pseudonyms.

The Sensitive DB contains five tables, each serving a distinct role. Figure 4.11 depicts Sensitive DB's physical model.

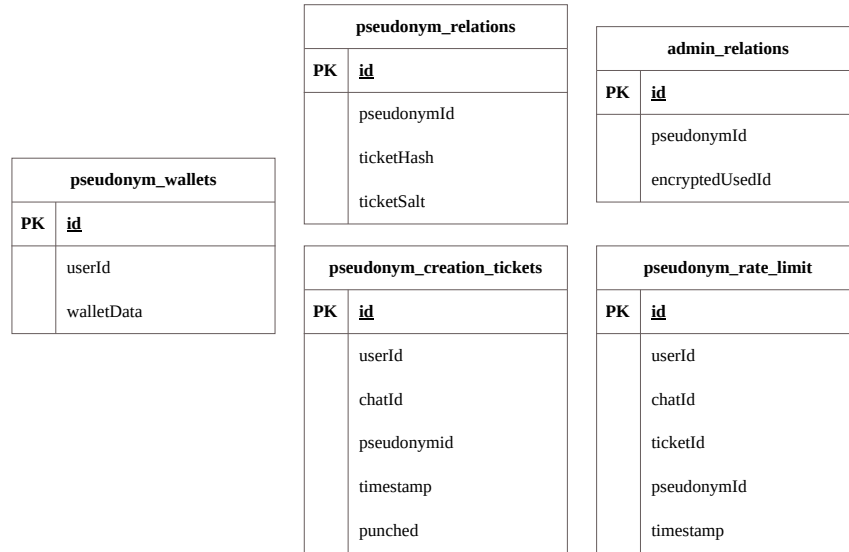


Figure 4.11: Physical Data Model of the Sensitive DB

The security mechanisms behind these tables, including encryption and ticket-based ownership, are described in more detail in the Anonymity section 6.2.8.

Chapter 5

Frontend

The Frontend Application is set of Flutter based applications for mobile devices (Android and iOS) and modern web browsers. The Frontend Application connects to the Backend Services in order to provide the user the ability to execute actions in the System and be able to see their result. Actions done by other users in the system can also reflect on the information shown to the user without an explicit action by processing server sent events.

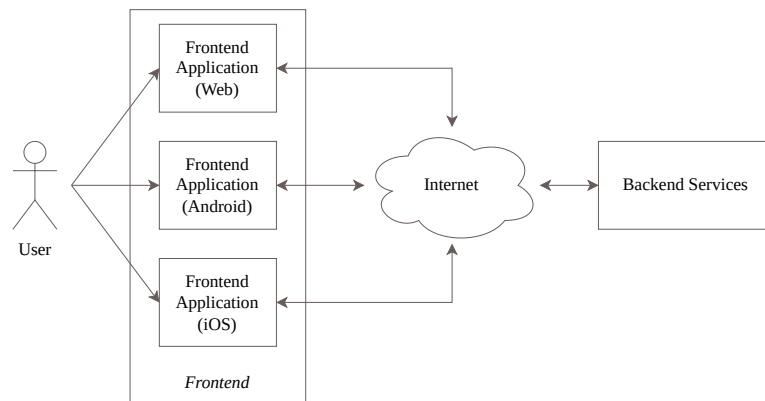


Figure 5.1: Frontend Interactions

5.1 Technology

The chosen framework for the Frontend Application is Flutter. This decision came after a research process, where several criteria were pondered for multiple frameworks as described in section 5.1.1.

5.1.1 Research

This section details the comparative analysis conducted among three prominent frameworks: Flutter, React Native, and Compose Multiplatform. The selection of an appropriate cross-platform framework was important for the success of the team's project, as the goal was to produce applications on multiple platforms with a single codebase to maximize code reusability. The incorrect choice of this framework would lead to:

- Increased development time due to the need for platform-specific code.
- Unsupported features, which could result in needing to change the API to accommodate the limitations of the chosen framework.

The framework comparison was based on several key criteria:

- Language
- Performance
- Supported Platforms
- Learning Curve
- Community Support / Maturity
- Integration Capabilities
- Development Environment
- Cost / Licensing

For the Supported Platforms, a minimum criteria exists of being able to support Web and at least one mobile platform (Android or iOS). To measure the Learning Curve, the team used a scale of Low, Medium and High. To measure Performance and Community support / maturity, a scale of Poor, Ok, Good and Great was used.

Flutter

Flutter is an open-source framework from Google with the proposed mission of creating experiences in multiple platforms from a single codebase. It was released in May 2017 with many community and Google supported and maintained packages. Google themselves use it for their high profile applications like Google Ads or Google Pay as well as automotive brands like BMW with their My BMW app. The language used for building Flutter applications is Dart, which allows for hot reloading during development through the use of a language Virtual

Machine while final artifacts focus on performance by being produced with native machine code depending on the targeted platform or transpiled to JavaScript or WebAssembly for web clients. Development is suggested on JetBrains IDEs (IntelliJ IDEA) or those based on them (Android Studio) but recommended on Visual Studio Code.

The development toolchain is available for all required platforms and architectures, with the exception of ARM64 where there is only official support for MacOS due to Apple Silicon, requiring extra effort for Linux and Windows. All expected platforms are covered by this framework, with no disclosed need to create platform specific code for basic UI elements.

Language	Dart
Performance	Great
Supported Platforms	Android, iOS, and Web
Learning Curve	Medium
Community Support / Maturity	Great
Integration Capabilities	Flutter SDK not officially available for all architectures
Development Environment	IntelliJ IDEA, Android Studio, and Visual Studio Code
Cost / Licensing	None, open-source

Table 5.1: Summary of Flutter as a Development Technology

React Native

React Native is the open-source framework made and used by Meta on their flagship applications like Facebook, Instagram and Messenger. Just like its non-Native counterpart, the development languages are either JavaScript or TypeScript, with the resulting code being JavaScript for use with the internal Hermes engine in cases of the business logic, while the declared JavaScript UI code is translated to the native UI elements for each platform (e.g. a View is translated as a UIView in iOS). Due to its already 10 years of development (released in March 2015), React Native has a mature ecosystem and a strong community, with much available information online and active contributions to popular packages or to the framework itself. This is the case of React Native Web, as it is an "extension" that is not built into the framework itself and enables the possibility of publishing for Web Browsers, translating UI code into Web elements and using the ReactDOM. The above could prove a problem as there is no guarantee that elements or features are stable across platforms, possibly leading to codebase fragmentation. While React Native itself is free for use and development on the recommended Visual Studio Code, the recommended build tools (Expo) pose an issue of cost and availability. Expo allows for the hot reload of apps on mobile devices but control would be lost as this is achieved by pushing a new version to a local server in the development machine that is then accessed by the Expo Application on the device. Publishing using this

tool can also incur in costs as the build is made on Expo owned servers with limits on free downloads and build time.

Language	JavaScript / TypeScript
Performance	Good
Supported Platforms	Android, iOS and Web
Learning Curve	Low
Community Support / Maturity	Great
Integration Capabilities	Recommended tools are powerful but we lose control
Development Environment	Android Studio and Visual Studio Code
Cost / Licensing	Open-source, but recommended tools can incur costs

Table 5.2: Summary of React Native as a Development Technology

Compose Multiplatform

Compose Multiplatform from JetBrains, the makers of the Kotlin language, is a framework that builds upon the work made on Jetpack Compose for Android, the recommended UI framework by Google for Android. It is the younger of the three, with its release in August 2021, which is reflected upon its comparatively weaker community and still developing ecosystem of libraries. The fact that the Web part of Compose Multiplatform is still labeled as alpha, was enough to stop the research on this framework in the early stages, as the team felt it was not compatible with the minimum requirement that was set for the supported platforms. For this reason, Performance and Integration capabilities are marked as not determined, as the team did not investigate them further.

Language	Kotlin
Performance	not determined
Supported Platforms	Android, iOS and Web (alpha)
Learning Curve	Medium
Community Support / Maturity	Growing
Integration Capabilities	not determined
Development Environment	IntelliJ IDEA and Android Studio
Cost / Licensing	None, open-source

Table 5.3: Summary of Compose Multiplatform as a Development Technology

Decision

The team decided upon Flutter for the framework of the Frontend Application. The available libraries seemed to satisfy the team's needs and it was the one that fully complied with the

minimum requirement of Web plus a mobile platform. The fact that there is no need for external tools for building or publishing artifacts if following the recommended steps from the official documentation were an extra that outweighed the required effort of learning a new language. Something the team also took into account was that this was the first time that any of the members had contact with the framework, a challenge that the team accepted.

5.1.2 Libraries

Provider

The Provider library is a state management solution for Flutter applications. It allows for the creation of a tree of Providers, which can be used to share data and functionality across the application without the need for passing data through constructors or using global variables. This is particularly useful in large applications where managing state can become complex. Figure 5.2 exemplifies the tree structure of Providers.

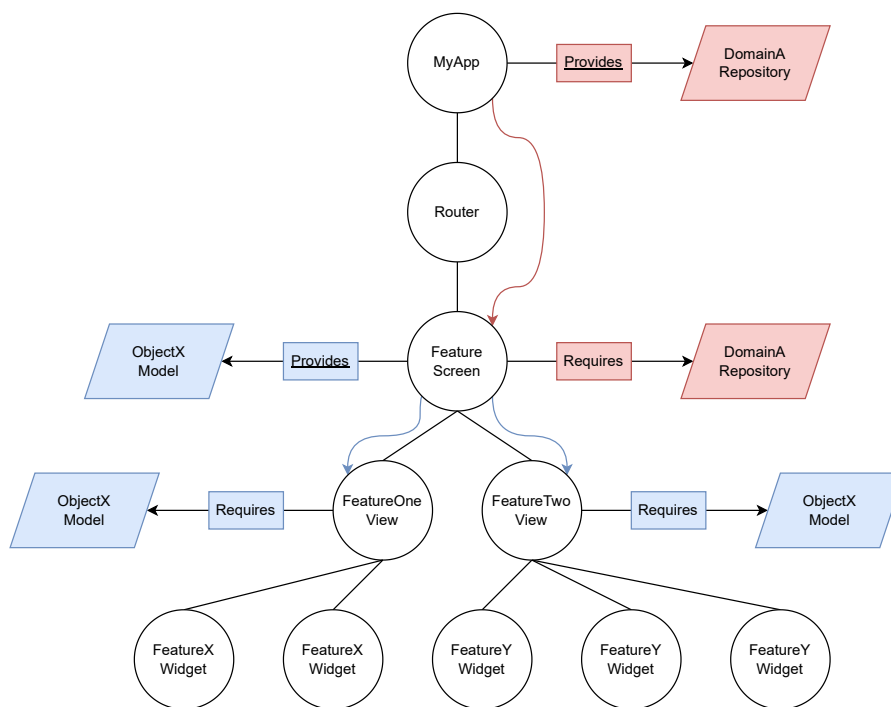


Figure 5.2: Example of the tree with Provider

ChangeNotifier for recomposition

ChangeNotifier is a class provided by the Flutter framework that allows for the notification of changes in the state of an object. It is used in conjunction with the Provider library to notify listeners when the state of a Provider changes, triggering a recomposition of the widgets that

depend on that Provider. Figure 5.3 shows an example of how `ChangeNotifier` is used to notify listeners of changes in the state.

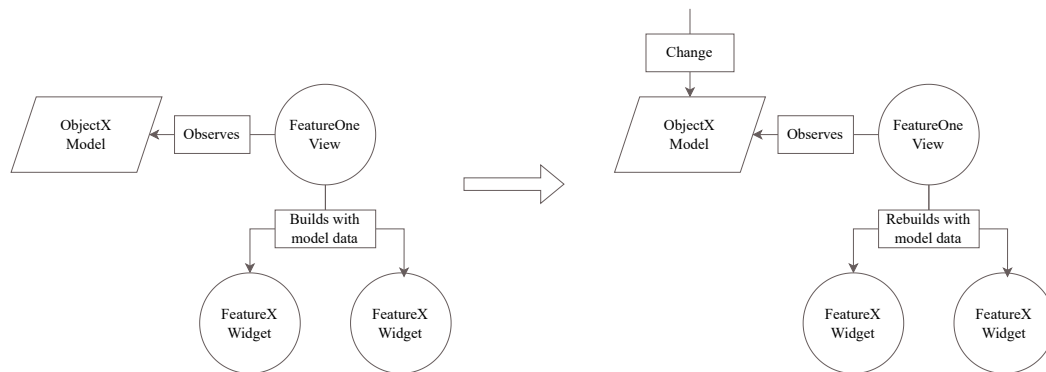


Figure 5.3: Example of `ChangeNotifier` for recomposition

Listing 5.1 shows an example of how `ChangeNotifier` is used in a class that manages a list of favorite chats. `FavoriteChatsModel` is a class that extends `ChangeNotifier`, allowing it to notify listeners when the list of favorite chats changes. The `addChatToFavorites` method adds a chat to the list and notifies listeners of the change.

```

1  class FavoriteChatsModel with ChangeNotifier {
2      List<FavoriteChat> _favorites = [];
3      List<FavoriteChat> get favorites => _favorites;
4
5      Future<FavoriteChatResult> addChatToFavorites(String
6          chatId) async {
7          final result = await _chatRepository.favoriteChat(
8              chatId: chatId);
9          return result.fold(
10             onFailure: (err) {
11                 return err;
12             },
13             onSuccess: (chat) async {
14                 _favorites.add(chat);
15                 notifyListeners();
16                 return FavoriteChatResult.chatAddedToFavorites;
17             },
18         );
19     }
20 }

```

Listing 5.1: Class extends ChangeNotifier

Listing 5.2 shows how a widget can observe the FavoriteChatsModel using the Consumer widget from the Provider library. The Consumer widget listens for changes in the FavoriteChatsModel and rebuilds the widget when the list of favorite chats changes, the change is triggered by the notifyListeners method in the FavoriteChatsModel class.

```
1      return ListView(  
2          children: [  
3              Consumer<FavoriteChatsModel>(  
4                  builder: (context, favoriteChatsModel, child) {  
5                      final favorites = favoriteChatsModel.favorites;  
6                      return Column(  
7                          children:  
8                              favorites  
9                                  .map(  
10                                     (favorite) => buildFavoriteChat(  
11                                         context,  
12                                         favorite,  
13                                         isMobileDevice,  
14                                         favoriteChatsModel,  
15                                     ),  
16                                  )  
17                                  .toList(),  
18                          );  
19                      },  
20                  ),  
21          ],  
22      );
```

Listing 5.2: Widget observes FavoriteChatModel

5.2 Organization

The Frontend Application is organized to clearly separate components that the user can interact with from the logic for interacting with the local device storage or the Backend Services.

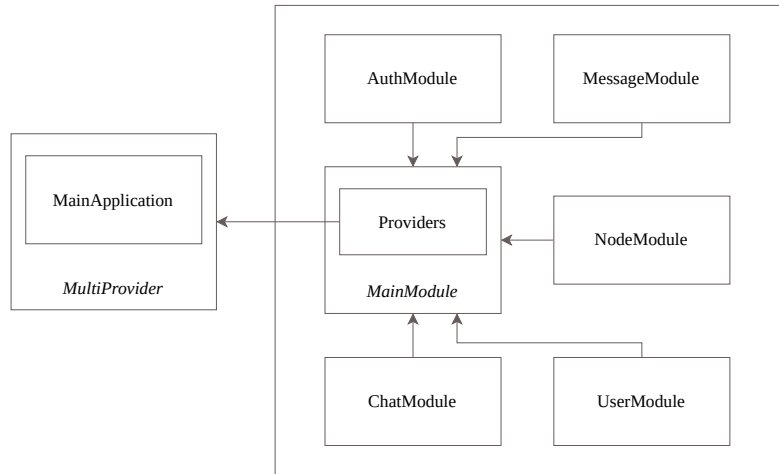


Figure 5.4: Dependency management in Main Application

The entrypoint of the Frontend Application prepares all required global scope dependencies by calling the `main_module` of the `di` (dependency injection) package. This module groups all of the domain modules, initializing each separately, so that any special cases of injection can be dealt with (e.g. providing required callbacks between modules for networking client).

To build our navigation tree, a router was used to declare the pages that the user can access. All paths accessible to the user for navigation and the components or rules enforced in each are declared here. Navigation is explained in section 5.3.

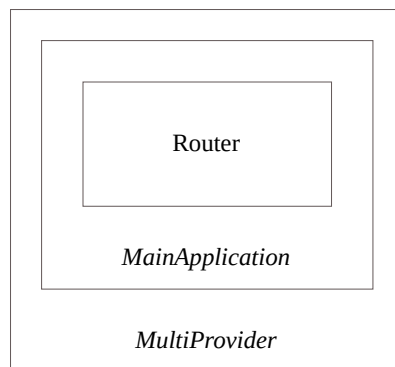


Figure 5.5: Router access to dependencies

All the necessary dependencies for the router and the UI that will be created upon navigating are available as it is a child of the `MainApplication`.

The convention the team followed for the UI components was: `Screen`, `View` and `Widget`.

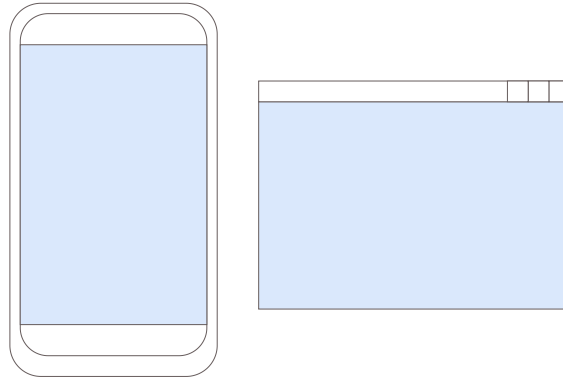


Figure 5.6: Content zone of a Screen

A Screen is a component that is normally thought of as a whole page as seen in figure 5.6, and tied to a path. If it is easier for modularity, a Screen can be composed of several Views.

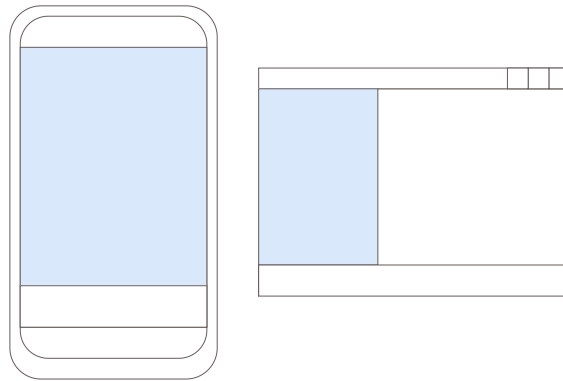


Figure 5.7: Content zone of a View

A View is a section of the page. It can be composed of multiple different Widgets, like in the case of a chat window that contains multiple messages and the input box to send a new one. Figure 5.7 shows an example of a View.

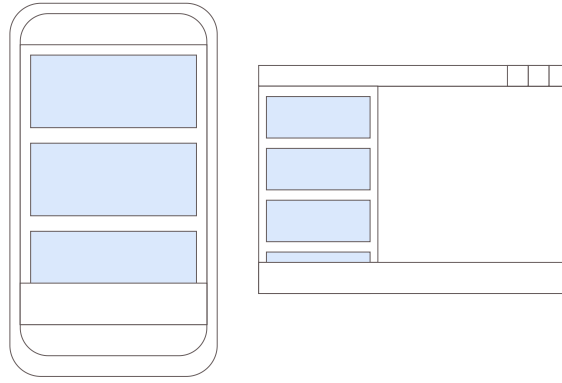


Figure 5.8: Content zone of multiple Widgets of the same type

A Widget is a lowest possible component, normally stateless and with emphasis on reusability inside of a View or a Screen. Figure 5.8 shows an example of a Widget that can be used to display a message in a chat window.

For the data layer, the team followed the recommended structure from the Flutter documentation of having a Repository as a way for the business logic to access data, and Services inside of those serving as the sources. Typically, a Repository that will store data on the local device and interact with the Backend Services will use a LocalService and a RemoteService as can be seen in figure 5.9.

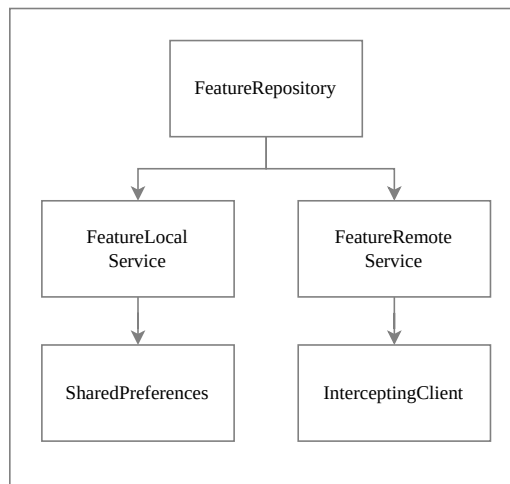


Figure 5.9: The structure of a common Repository

5.3 Navigation

The navigation system of the Frontend Application is designed to provide a consistent user experience across different platforms, allowing users to easily access various features and functionalities. Figure 5.10 illustrates the navigation structure of the Frontend Application.

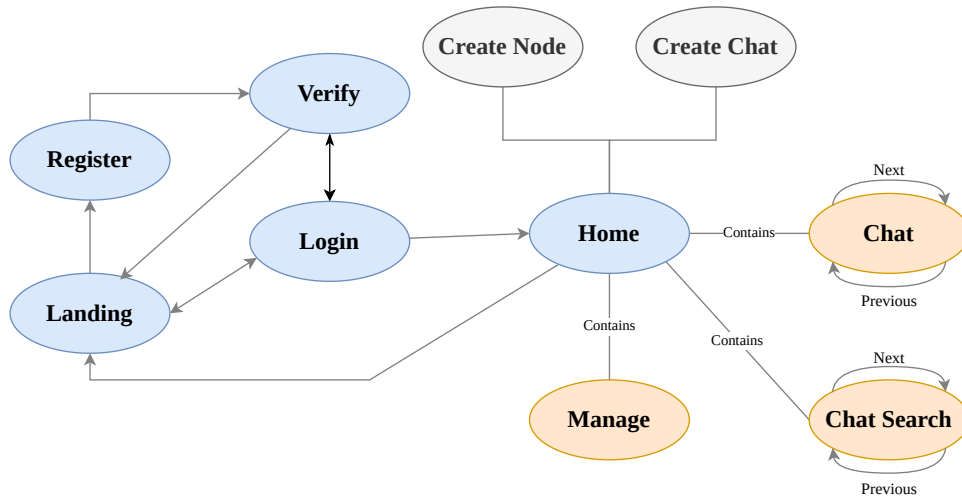


Figure 5.10: Navigation overview

The blue elipses represent the different screens that the user can access, the orange elipses represent the views that are part of those screens whilst the grey ellipses represent modals that can be opened on top of the screens. The arrows indicate the navigation paths between the screens, views, and modals, showing how users can move from one part of the application to another.

5.4 Networking

The `InterceptorClient` is a custom HTTP client that extends the functionality of the default HTTP client provided by Flutter. It is designed to handle network requests and responses, allowing for the interception of requests and responses to add custom headers, log requests, or handle errors in a centralized manner. Figure 5.11 illustrates the flow of network requests and responses through the `InterceptorClient`.

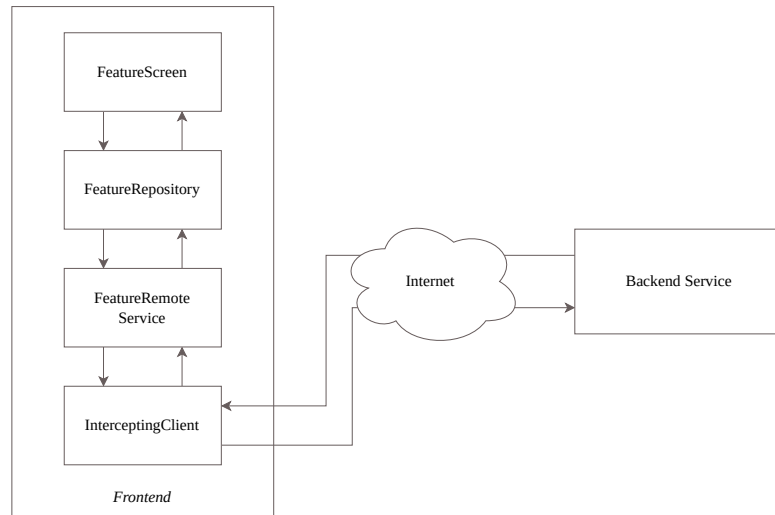


Figure 5.11: InterceptorClient use

5.5 Platform Specific Code

The Frontend Application is designed to run on multiple platforms, including Android, iOS, and Web. Flutter allows for platform-specific code to be written and called from the shared codebase; however, during the development there was no need to write any platform-specific code to call platform-specific APIs, as the system was able to run on all platforms using the shared codebase.

There was however the need to write code due to screen size differences between platforms. Due to the smaller screen sizes of the mobile platforms, the NodeTree widget wasn't correctly displayed, causing the application to be unusable as it was occupying screen space that was supposed to be used for the partial views. In order to circumvent this issue, a media query was used to determine the screen size and display the NodeTree widget only on larger screens, while smaller screens implemented a Drawer to display the NodeTree widget. This way, the NodeTree widget is only displayed when there is enough screen space to do so, allowing the application to be usable on all platforms.

```

1  bool isMobile(BuildContext context) =>
2  MediaQuery.of(context).size.width < 800;

```

Listing 5.3: Media Query

```

1  @override
2  Widget build(BuildContext context) {
3      final isMobileDevice = isMobile(context);
4      return Scaffold(

```

```

5      drawer: isMobileDevice
6        ? Container(
7          color: Colors.grey[100],
8          child: const NodeTree(),
9        )
10       : null,
11      appBar: isMobileDevice ? AppBar(
12        title: const Text(''),
13        leading: Builder(
14          builder: (context) => IconButton(
15            icon: const Icon(Icons.menu),
16            onPressed: () {
17              Scaffold.of(context).openDrawer();
18            },
19          )
20        ),
21      )
22      ...
23    );
24  }

```

Listing 5.4: Drawer creation based on Media Query result

5.6 Testing

To ensure the reliability of the Frontend Application, the team implemented several types of tests commonly used in Flutter apps. In Flutter, the following testing techniques are available:

- **Unit Testing:** tests individual units of code, such as functions or methods, in isolation.
- **Widget Testing:** tests widgets by rendering them in a controlled environment to verify their behavior and appearance.
- **Integration Testing:** tests the interaction between multiple widgets and the app's interaction with external services. These tests simulate real-world usage scenarios and are more complex, often involving interactions with the UI, API calls, and databases.

5.6.1 Unit Testing

To perform the simplest testing technique, the team used Flutter’s built-in testing framework `flutter_test` and `mocktail`, a mocking library for simulating dependencies.

5.6.2 Widget Testing

Widget Testing verifies how widgets render and behave. It is better than unit testing for UI because it tests layout and interactions. For this technique, the team used `flutter_test` and `mockito`.

5.6.3 Integration Testing

For integration testing, the team used Maestro, a UI testing framework that simplifies test automation through human-readable YAML scripts. Maestro supports cross-platform testing (mobile and web) and provides fast execution with minimal setup. Maestro Studio, its visual interface, helps build tests by interacting with the app in real time and exporting test flows as YAML.

Listing 5.5 illustrates an example of a YAML script used by Maestro, where an application is automatically launched and the login process begins.

Maestro provides full support for Flutter apps by interacting with widgets via semantics labels from Flutter’s Semantics widget. By default, it can access widgets like `Text` and `TextField` through their displayed text or `hintText`.

In the Dart code of the listing 5.6, the `ElevatedButton` widget is wrapped around the widget `Semantics`, allowing for a label named `'login_button'` to be used in the YAML script of listing 5.5.

```
1 appId: com.example.confidefrontend
2 ---
3 - launchApp
4 # Assert landing screen shows expected text
5 - assertVisible: "Welcome to Confide!"
6 - assertVisible: "Register"
7 - assertVisible: "Login"
8
9 # Navigate to Login screen
10 - tapOn: "Login"
11
12 # Tap login button without entering data
```

```

13 - tapOn: 'login_button'
14
15 # Assert that the error is shown
16 - assertVisible: "Email is required."

```

Listing 5.5: Maestro YAML script

```

1
2 Semantics(
3   label: 'login_button',
4   child: ElevatedButton(
5     onPressed: _onButtonPress,
6     child: const Text('Login'),
7   )
8 )

```

Listing 5.6: Use of label from Flutter's Semantics widget

Flutter web apps / web pages running in a desktop browser are supported by Maestro, though they have some limitations, such as the lack of multi-browser support and screen size configuration. Flutter Desktop apps, however, are not yet supported.

5.6.4 Differences

In table 5.4, it is possible to visualize the advantages and disadvantages of each test technique.

Test Type	Advantages	Disadvantages
Unit Testing	- Easy to write, maintain, and debug - Runs in isolation from Flutter framework - Encourages clean architecture (separation of concerns)	- Only validates logic, not UI or user flows - Limited coverage: cannot detect integration or layout issues
Widget Testing	- Validates widget rendering and behavior - Can simulate user interactions and layout behavior - Detects layout, animation, and rendering issues	- Cannot test end-to-end app behavior - May require mocks for dependencies - Less reliable when widgets have complex side effects or async logic
Integration Testing	- Simulates full user journey - Tests UI, navigation, API calls, and backend integration - Closest to real-world usage - Useful for regression testing and QA validation	- Slowest and flakiest test type - Requires setup of environment, mocks, or test servers - Maintenance can be costly - May fail due to non-code-related issues (e.g., network delays)

Table 5.4: Comparison of Flutter Test Types

Chapter 6

Backend

This chapter presents the System's Backend architecture, the technologies used, important aspects and implementation decisions. The Backend is responsible for managing the business logic, data storage, and communication with the Frontend. The architecture is illustrated in figure 6.1.

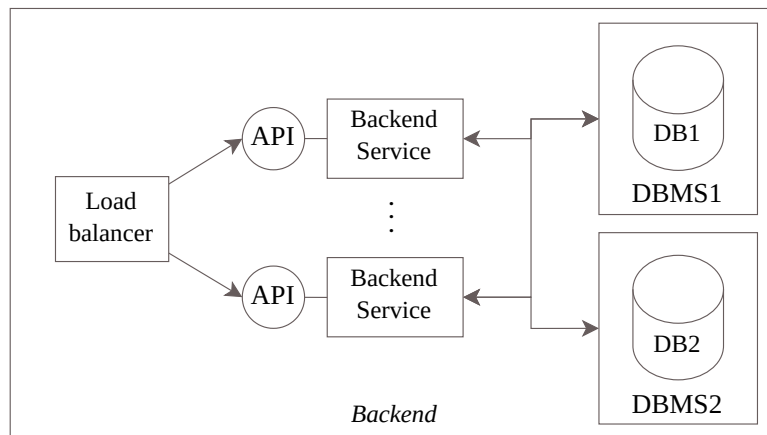


Figure 6.1: Backend Architecture

6.1 Load Balancer

The Load Balancer is the component responsible for distributing incoming requests across multiple servers. This ensures that no single backend service becomes overwhelmed with traffic, improving performance and reliability. The technology implemented for the Load Balancer is *Nginx*. The reason for this choice is that the team had previous experience with the technology, and it is a well-known and widely used solution for load balancing.

6.2 Backend Services

The Backend Services expose HTTP endpoints that support the realization of domain operations. The Backend Services are responsible for handling incoming requests from the Frontend, processing them, and returning appropriate responses. Figure 6.2 illustrates the architecture of a Backend Service.

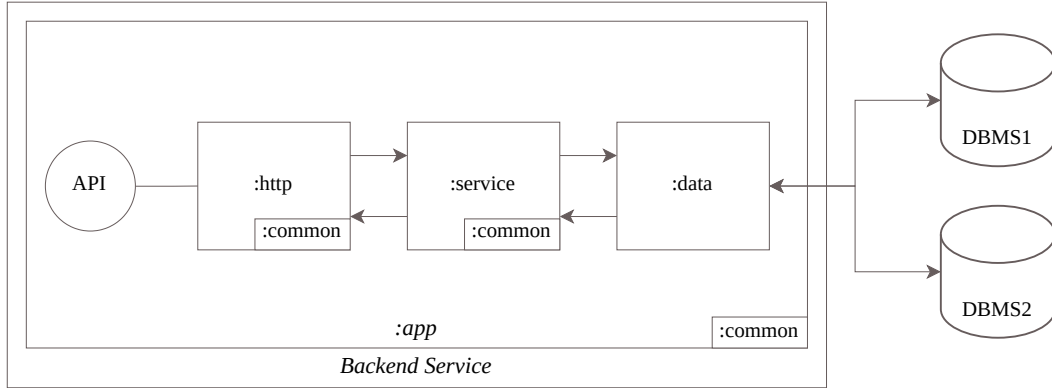


Figure 6.2: Backend Service Architecture

The arrows in figure 6.2 represent the flow of information between the different layers of the Backend Services. The Controller layer receives and processes incoming requests and delegate them to the service layer. The Service layer processes the requests, implementing the System's business logic and interacts with the Repository layer to perform operations. The Repository layer interacts with the Databases to store and retrieve data.

6.2.1 Technology

The selected framework for the implementation of the Backend Service is *Spring Boot*. The team had previous experience with the framework, which made it a natural choice for this project.

6.2.2 Organization

The Backend Service is implemented in a way which emphasizes separation of concerns and modularity. The project is structured as a multi-module Gradle project, where each module is defined as a separate Gradle subproject with its own dependencies and responsibilities. This ensures a strict separation of concerns and helps prevent unnecessary dependencies between modules. Gradle's build configuration and dependency management play a key role in enforcing this modular architecture, leading to better organization of the codebase and improved maintainability.

Figure 6.3 illustrates the dependency graph of the Backend Service. Arrows represent dependencies and visibility between modules, with the direction of the arrow indicating the direction of the dependency and the visibility. If a module depends on another, it points to the module it depends on. If there is no arrow, it means that the module is not visible to the other module.

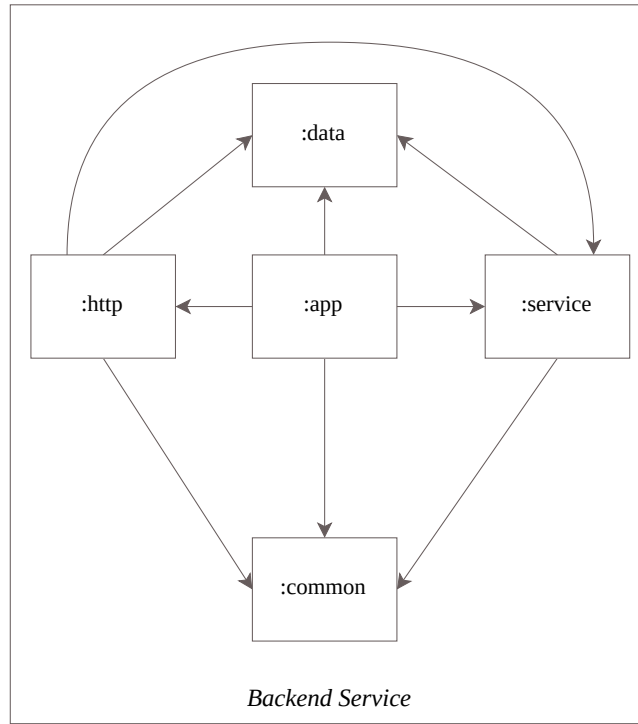


Figure 6.3: Backend Service Organization

- **App module:** This module contains the main application class and configuration files. It is responsible for initializing the application and configuring the necessary components, such as the database connection and security settings. It is not visible to other modules and it depends on the HTTP, Service and Data modules.
- **HTTP module:** This module is responsible for handling incoming HTTP requests and returning appropriate responses. It acts as the entry point for the application, receiving requests from the Frontend and delegating them to the appropriate service. It is visible to the App module and it depends on the Service, Data and Common modules.
- **Service module:** This module contains the business logic of the application. It processes requests from the Controller layer, interacts with the Data layer, and returns results to the Controller. It is visible to the App module and it depends on the Data

and Common modules.

- **Data module:** This module is responsible for data access and storage. It interacts with the Databases and provides an abstraction layer for the Service layer to perform CRUD operations. It also contains the data models used by the application. It is visible to the App, HTTP and Service modules and has no dependencies on other modules.
- **Common module:** This module contains common classes and utilities that are used in the app, HTTP and service layers. It includes configuration classes, constants and validators. It is visible to the HTTP and Service modules and has no dependencies on other modules.

6.2.3 HTTP API Endpoints

The HTTP module exposes endpoints for each entity of the System, allowing the Frontend to interact with the Backend. The endpoints are organized by entity, with each entity having its own controller class. For a detailed list of the endpoints, see section 9.2.

6.2.4 Authentication

The user authentication is handled by the Backend Service. The team opted not to use Spring Security for authentication as it would add unnecessary complexity to the project. Instead, a custom authentication solution was implemented, http-only authentication cookies. Initially the team implemented JWT (Json Web Tokens) as they would be able to transport the user's pseudonym wallet, but in the end opted out of this implementation. Due to the way that Anonymity was implemented, the System always has to query the database, therefore there was the same amount of database queries as other authentication methods. It also ended up not making sense to pass the user permissions through the JWT, due to the size and flexibility of the permissions. The benefits of having a JWT weren't being taken advantage of, and for that reason, the team moved away from that implementation.

The implemented authentication method uses http-only cookies to store the user's session on a valid login. Since it is the Backend Service that sets the cookie, the Frontend Applications then need to manage the session state according to the running platform.

6.2.5 Permissions

The Backend Service implements a custom solution for user permissions, which is based on the Group entity. Although there are no predetermined roles in the System, a role can be defined by creating a group with the desired permissions and assigning the users to it. The Group entity is responsible for managing the System's user's permissions. Each user can

belong to multiple groups, and each group can have multiple users. Each group can have multiple nodes, and each node can have multiple groups. This allows for a flexible and scalable permission system, where users can have different permissions in different nodes.

6.2.6 Server Sent Events

The Backend Service uses Server Sent Events (SSE) to push real-time updates to the Frontend. SSE is a standard for server-to-client communication that allows the server to send updates to the client over a single HTTP connection. This is particularly useful for applications that require real-time updates, such as the System described. The System provides real-time updates for the following events:

- **New Message:** When the user is actively participating in a chat and a new message is sent in this chat, the Backend Service pushes the message to clients who are listening. This event is also used to increment the number of unreadMessages of chats that the user has favorited, but are not open in the Frontend application. This allows the user to see the last message of the chat and the number of unread messages without having to constantly poll the server for updates.
- **Message Deleted:** When a message is deleted in a chat, the Backend Service pushes an event to clients who are listening. This allows the Frontend to update the message from the chat view in real-time.
- **Message Reported:** When a message is reported in a chat, the Backend Service pushes an event to clients who are listening. This allows the Frontend to update the message from the chat view in real-time, marking it as reported.

Technology used for SSE

The technology chosen to support the implementation of SSE was *Redis Pub/Sub* [13]. Redis Pub/Sub is a messaging system that allows backend services to publish messages to specific channels. In this architecture, the Backend Service uses Redis as a message broker to publish messages when a new chat message is sent or a notification is created. The Backend Service exposes an endpoint that subscribes to these Redis channels and forwards relevant messages to connected Frontend clients over an HTTP-based SSE connection. This enables real-time updates to be pushed to the Frontend without the need for polling, while maintaining a clear separation between Backend Services and Frontend clients. The team also considered using PostgreSQL's LISTEN/NOTIFY feature, but decided against it as Redis Pub/Sub provided a solution that would allow the System to scale better in the future.

Redis Pub/Sub proved to be a very powerful tool and the team ended up using it for other features of the System, mainly for the optimization of the System's initial setup and the Unveiling Identity feature. The following features were implemented using Redis Pub/Sub:

- **Backend Service synchronization:** The Backend Services use Redis Pub/Sub to synchronize setup state, allowing launched Backend Services to receive the current system state instead of polling it from the database
- **System start-up key-share:** The Backend Services use Redis Pub/Sub to share created public key among themselves, preventing I/O from being done every time the key needs to be used.
- **Unveiling Identity:** The Backend Services use Redis Pub/Sub to share the Administrator's key share across Backend Services, allowing one to reconstruct the key and unveil a pseudonym's true identity.

6.2.7 Database Access

Every service of `:service` has Transaction Managers injected into them, which are responsible for managing the transactions, ensuring that the operations are atomic and consistent. The signature of the Transaction Manager is shown in listing 6.1.

```
1 interface TransactionManager<T : Transaction> {
2     fun <R> run(block: (T) -> R): R
3 }
4
5 abstract class JdbiTransactionManager<T : Transaction>(
6     private val jdbi: Jdbi,
7 ) : TransactionManager<T> {
8     protected abstract fun createTransaction(handle: Handle
9         ): T
10
11     override fun <R> run(block: (T) -> R): R =
12         jdbi.inTransaction<R, Exception> { handle ->
13             val transaction = createTransaction(handle)
14             block(transaction)
15         }
16 }
```

Listing 6.1: Transaction Manager Signature

The DataTransaction and SecureTransaction Interfaces contain all of the Repositories that the Services need to access the Databases. The signatures of the Interfaces are represented in listings 6.2 and 6.4 respectively. The DataTransaction Interface contains all of the Repositories that are used to access the main application database, while the SecureTransaction Interface contains the Repository used to access the sensitive data database, ensuring that started transactions are properly isolated and that the data is accessed in a consistent and safe manner. To perform a transaction within the Backend Service, the Service layer calls the run method of the specific Transaction Manager, passing a block of code that contains the operations to be performed within the transaction. The Transaction Manager then creates a new transaction, executes the block of code, and commits or rolls back the transaction. The Transaction managers can be seen in listings 6.3 and 6.5. The DataTransactionManager is responsible for managing transactions for the main application database, while the SecureTransactionManager is responsible for managing transactions for the sensitive data database.

```
1 interface DataTransaction : Transaction {
2     val chatRepository: ChatRepository
3     val nodeRepository: NodeRepository
4     val authenticationRepository: AuthenticationRepository
5     val configurationRepository: ConfigurationRepository
6     val userRepository: UserRepository
7     val messageRepository: MessageRepository
8     val groupRepository: GroupRepository
9     val permissionRepository: PermissionRepository
10    val setupRepository: SetupRepository
11 }
```

Listing 6.2: Data Transaction Interface Signature

```
1 class JdbiDataTransactionManager(
2     @Qualifier("dataJdbi")
3     jdbi: Jdbi,
4 ) : JdbiTransactionManager<DataTransaction>(jdbi) {
5     override fun createTransaction(handle: Handle):
6         DataTransaction = JdbiDataTransaction(handle)
7 }
```

Listing 6.3: Data Transaction Manager

```
1 interface SecureTransaction : Transaction {
2     val secureRepository: SecureRepository
```

```
3      }
```

Listing 6.4: Secure Transaction Interface Signature

```
1      class JdbiSecureTransactionManager(  
2          @Qualifier("secureJdbi")  
3          jdbi: Jdbi,  
4      ) : JdbiTransactionManager<SecureTransaction>(jdbi) {  
5          override fun createTransaction(handle: Handle):  
6              SecureTransaction = JdbiSecureTransaction(handle)  
      }
```

Listing 6.5: Secure Transaction Manager

6.2.8 Anonymity

The Backend Service implements a custom solution for user anonymity, designed to allow users to interact in chats without their identity being revealed. The System promises to keep the user’s identity hidden, even if the following occurs:

- Content Database Leak
- Server Compromise
- Bad Faith Actors

Upon sending a message in a chat, if the user has the "be_pseudonym" permission, the Backend Service generates a One Time pseudonym creation ticket, along with a pseudonym and ownership proof. This information must be returned to the Backend Service in order to complete the pseudonym creation process. In case the user does not receive the information, the Backend Service provides a fallback mechanism, allowing the user to request a new pseudonym but still using the same One Time ticket. This ensures that the user can always create a pseudonym, even if the initial request fails. Once a pseudonym is confirmed, the One Time ticket is punched, meaning that it can no longer be used to create a new pseudonym. This prevents the user from creating multiple pseudonyms for the same chat, and allows the system to know that the user has a pseudonym in that chat. The confirmation of a pseudonym also creates a relation between the pseudonym and the ciphered userId, allowing the Backend Service to decipher the user’s identity, given the admin’s reconstructed key. This is done by storing the relation in the `admin_relations` table. The Sensitive DB contains the needed information to prove ownership of a pseudonym, to unveil the user’s identity and to retrieve the user’s pseudonyms through his Wallet. The wallet should be encrypted by the frontend

client, making it unreadable by the Backend Service and allowing the user to synchronize his pseudonyms across multiple devices.

Since all the information related to the user's identity is stored in hashed or encrypted format, the user's identity is protected even in the event of a server compromise. Bad actors are not able to brute force the hashed information, as every Id used in these combinations is an UUID. This means that the number of possible combinations is extremely large, making it infeasible to brute force the information. The only way to retrieve the user's identity is to have access to the private key of the Backend Service or to have access to the user's device. Since the System's private key is a Shared Secret (assured by Shamir's Secret Sharing, [21]), even an administrator acting in bad faith is not able to decipher the user's identity.

6.3 Databases

The technology used for the databases is *PostgreSQL*. PostgreSQL is an open-source relational database management system, which the team had previous experience with. The System uses two databases, one for the main application and another for the sensitive data related to user identities.

The main application database is used to store all the data related to the application, such as messages, chats, users, etc. The sensitive data database is used to store information related to user identities. The databases are managed by two instances of PostgreSQL, which are configured to run on different ports. This ensures the physical split of the databases.

Chapter 7

Infrastructure

This chapter provides an overview of the infrastructure that supports the development and deployment of the System applications, using a combination of cloud services and local resources.

7.1 Overview

The team built a robust infrastructure to support the development and deployment of the System applications. This infrastructure includes CI/CD pipelines along with a Cloud Provider that allows the team to deploy applications in a scalable manner. Figure 7.1 provides an overview of the infrastructure architecture.

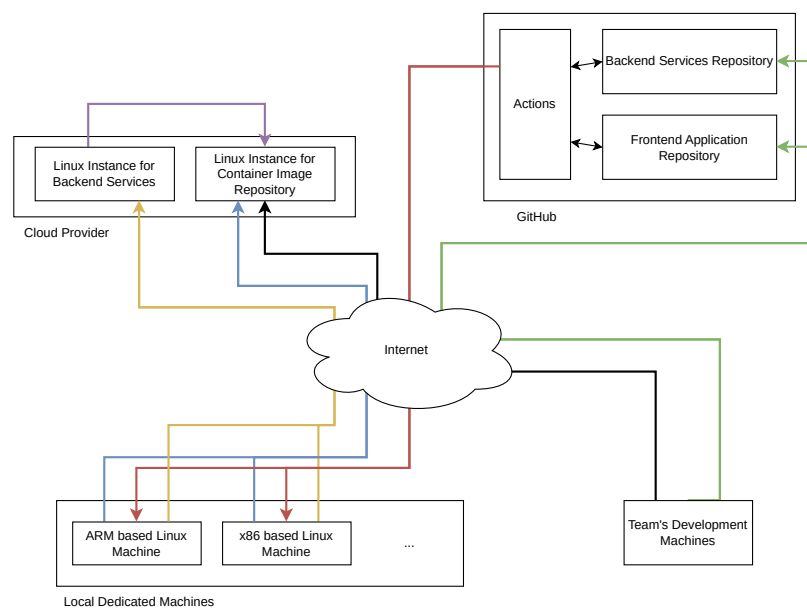


Figure 7.1: Infrastructure overview

7.2 Cloud

The team utilized DigitalOcean [3] as the cloud provider for hosting the System applications. DigitalOcean offers a range of services that are well-suited for deploying and managing applications in a cloud environment, including virtual machines and managed databases.

7.3 Local

This section describes the local infrastructure used by the team to support the development and testing of the System applications. The local setup includes machines for development, automated configuration for reproducibility.

7.3.1 Machines

As part of the local infrastructure, the team utilized a combination of personal machines, one ARM machine, and one x86 machine. These machines provided runners for the Github CI/CD pipelines, allowing the team to test all changes before merging them into the main branch. These runners poll Github for jobs and execute them, allowing the team to freely utilize Github Actions without incurring additional costs. These machines were also used as one way communication channel, as they populated Github events such as pull requests and issues to the team's Discord server. This setup allowed the team to stay informed about the project's progress and changes in real-time, facilitating better collaboration and communication among team members.

7.3.2 Automated configuration

To ensure that the local development environment is reproducible, the team used Docker. Docker allowed the team to create seeded Databases that allowed the continuous integration pipelines to run tests in a consistent environment. This setup ensured that the integration tests could be run reliably, regardless of the local machine's configuration.

7.4 GitHub

The team used GitHub [8] as the primary platform for version control and collaboration. GitHub provided a robust environment for managing the source code, tracking issues, and facilitating collaboration among team members. The team took advantage of GitHub Actions to implement pipelines, which automated the process of building, testing, and deploying the System applications. Certain Github events were also defined to notify the team members on

Discord, keeping them informed about changes and updates in the project. This integration allowed for efficient development workflows and ensured that code changes were thoroughly tested before being merged into the main branch. The use of GitHub also enabled the team to maintain a clear history of changes, manage pull requests, and conduct code reviews, further enhancing the quality and reliability of the software development process.

Chapter 8

Conclusion

This report has presented the development of a multi-platform application that aimed to provide a safe place for students to share their doubts and questions while remaining anonymous. The main requirements of the System were to ensure user anonymity, provide a configurable solution so that it could be adapted to different educational institutions, and to be accessible on multiple platforms. The System was developed using the Flutter framework for the Frontend and a Nginx, Spring Boot and PostgreSQL stack for the Backend. The choice of technologies was made based on the team's expertise and the requirements of the System.

8.1 Achievements

The delivered System supports an onboarding process, allowing System Administrators to do the initial configuration of the system, providing the first users, base Category name and allowed email domains. User permissions are managed through Groups, which are created by the System Administrators or Teachers, allowing them to assign users to specific groups and manage their permissions. Groups have users, categories and permissions associated with them, allowing fine control over what users can do within each Category. Categories can be created, as well as chats within a category. Users that can see the category can read its chats without actually joining them. Chats can be favorited, allowing users to quickly access their favorite chats. Users can interact with each other in chats and report messages they think don't follow the guidelines. The System supports real-time updates through Server-Sent Events (SSE), allowing users to receive updates on new messages, unread messages and message state updates in real time. Moderators can manage the reported messages, verifying them (clearing the reports) or effectively deleting them. Users are automatically assigned a Pseudonym if they have that permission, allowing them to act as a Pseudonym within that specific chat, remaining anonymous. A Pseudonym's true identity can be unveiled with the explicit action of two out of three System Administrators, allowing for a balance between

anonymity and accountability.

The Frontend Applications use a single code base, from which it is possible to create different artifacts. For mobile platforms, artifacts with native code are generated, while for the web platform, artifacts are generated in JavaScript. The Backend Services were developed with horizontal scaling in mind, meaning multiple instances can be run in parallel, and the Load Balancer will distribute the requests among them, increasing System availability. Due to this design, Redis Pub/Sub was used to ensure that all instances of the Backend Services receive the same messages, allowing for real-time updates across all instances. Redis Pub/Sub was used not only for SSE events across the multiple Backend Service instances, but also to synchronize needed state between them, such as Administrator private key shares to allow for the unveiling of a Pseudonym's true identity. SSE connections are handled as a "all or nothing" approach, meaning that if a user has an active SSE connection, they will receive all events all of their favorite chats and the chat they currently have open. If there is any change in user interest, like moving to a different chat room, the connection is closed and a new one is opened, aligning with the fact that there is more than one Backend Service instance running at the same time, meaning that a user connection is always handled by one Backend Service instance, the responsibility is not shared among them.

8.2 Improvements and Future Work

While the System has achieved its main objectives, there are still areas for improvement and future work. Some of the improvements and future work include:

- Finalize the Wallet implementation, making it ciphered data in the Backend Service and allowing users to retrieve their Pseudonyms using the Wallet and deciphering the data in the Frontend Applications. This will allow the user's identity to be kept safe and always hidden from the Backend Service and bad actors.
- Implement identity unveiling on the Frontend Applications, allowing users to unveil a Pseudonym's true identity from the Frontend Applications without having to use the Backend Service directly.
- Implement more events for the SSE connections, allowing users to receive more real-time updates, such as Category and chat updates, permission changes and others.
- Fix the bug related to Router caching, which causes the Frontend Applications to have inconsistent behaviour when pressing the back and forward buttons to navigate.
- Implement a better way to handle the user's wallet. The current implementation is not bulletproof as if the user loses their pin, they will lose access to their wallet and the

tickets associated with it. A recovery mechanism could be implemented, invalidating previous pseudonyms and allowing the user to create a new wallet and pseudonyms.

- Implement a clean up mechanism for the Backend Service. In order to support communication failures when requesting a Pseudonym, the Backend Service stores data that can become unusable, which can deteriorate database performance.
- Additional features could be added such as support for read/unread messages, spoiler messages (to view reported messages) and others.

8.3 Challenges Faced

8.3.1 Non-technical Challenges

During the initial phase of the project, the team faced several non-technical challenges.

Excessive Focus on documentation

One of the main challenges was the exaggerated focus on technical aspects and documentation at the expense of starting the development with a simpler version of the System. The team spent a significant amount of time discussing and documenting technical details, which delayed the actual development process. This led to a situation where the team had a lot of documentation but little progress in terms of a working prototype.

Organizing work

Another challenge was the planning and organization of the work process. The team struggled to find a balance between planning and execution, which delayed the development of the System. The team realized that they needed to focus on building a working prototype first, and then iterate on it based on user feedback and testing.

Adjusting to a New Approach

The challenges faced reinforced the need for a more agile approach to development, where the team could quickly build and test a working prototype, gather feedback, and then iterate on it. This approach would allow the team to focus on delivering value while also ensuring that the technical aspects of the System were well-documented and maintained.

8.3.2 Technical Challenges

The development of the System also presented several technical challenges.

Flutter

One of the main challenges was the framework choice for the Frontend Applications. After investigation and accepting the challenge of researching and learning a new framework from scratch, the team settled with Flutter. The team faced a learning curve with Flutter, as it was a new framework for all the members. However, the team was able to overcome this challenge by dedicating time to learning and experimenting with the framework, which ultimately paid off in terms of code reuse across different platforms. The documentation and community support for Flutter were also helpful in overcoming the technical challenges faced during development.

SSE implementation

Another technical challenge was the implementation of Server-Sent Events (SSE) for real-time updates in the System. The non-functional requirement of system horizontal scalability added complexity to the SSE implementation, as it required careful consideration of how to handle multiple connections and ensure that updates were delivered efficiently to all connected clients. The team also had to learn and implement Redis Pub/Sub, which is a message broker, a technology that the team had not used before.

Anonymity

A significant challenge faced by the team was ensuring user anonymity while still providing a working system. For the System to be truly safe, against all of the threats that could compromise the anonymity of the users, the team tried to implement Zero Trust Architecture as much as possible, along with other techniques. This meant that the System had to be designed in a way that the client could never be trusted, which added complexity to the development of the anonymity features.

8.4 Lessons Learned

The development of the System provided several valuable lessons for the team.

8.4.1 Importance of Prototyping

One of the main lessons learned was the importance of building a working prototype early in the development process. An early and iterative working prototype would allow the team to gather feedback and iterate on the design and functionality of the System, rather than getting bogged down in documentation and technical details.

8.4.2 Agile Approach

The team also learned the value of adopting a more agile approach to development, where the focus is on delivering value quickly and iterating based on feedback.

8.4.3 Collaboration and Communication

Collaboration and communication within the team were also key to overcoming the challenges faced during development. The team learned the importance of regular check-ins, sharing knowledge, and supporting each other in learning new technologies and approaches.

8.4.4 Documentation

Finally, the team learned that while documentation is important, it should not come at the expense of actual development. The team realized that documentation should evolve alongside the System, rather than being a static artifact created at the start of the development process.

Chapter 9

Appendix

This chapter contains additional information that supports the main content of the report but is not essential to its general understanding. It includes supplementary material such as detailed data model explanations and any other relevant information that may be useful for readers who want to delve deeper into the project.

9.1 Detailed System entities

9.1.1 Content Database Entities

The following section provides a detailed description of the entities in the Content Database. Each entity is described with its attributes.

- **System State**

Represents the current state of the system. Attributes:

- `setup_complete` (boolean) — True if the system setup is complete, False otherwise.
- `state_id` (integer) — Unique identifier for the system state.
- `adminEmails` (string) — Comma-separated list of email addresses of system administrators.

- **System States**

Represents the valid states of the system. Attributes:

- `state_slug` (string) — A human readable identifier for the system state.
- `description` (string) — A description of the system state.

- **User**

Represents the users of the system. Attributes:

- **name** (string)
 - **email** (string)
 - **registerDate** (timestamp)
 - **banned** (boolean) — True if banned, False otherwise
 - **active** (boolean) — True if active, False otherwise
- **session_tokens** Represents the session tokens issued to users. Attributes:
 - **tokenValidation** (string)
 - **requestTime** (timestamp)
 - **lastUsed** (timestamp)
- **Pseudonym**
Represents the pseudonyms attributed to users upon joining a chat. Attribute:
 - **name** (string)
 - **used** (boolean) — True if the pseudonym is used, False otherwise
 - **timestamp** (timestamp) — The date and time the pseudonym was created. Enables cleanup.
- **Authentication**
Stores encrypted authentication information linked to registered users. Attribute:
 - **password** (string)
- **AuthorizedDomain**
Represents allowed email domain names for user registration. Attributes:
 - **domain** (string)
 - **isEnabled** (boolean) **True** if the domain name is enabled **False** otherwise
- **RegistrationToken**
Stores tokens issued upon user registration. Attributes:
 - **token** (UUID)
 - **requestTime** (timestamp)
 - **expiryTime** (timestamp)
 - **isValid** (boolean) - True if valid, False otherwise
- **password_reset_info**
Stores information about password reset requests. Attributes:

- `tokenValidation` (string)
- `requestTime` (timestamp)
- `expiryTime` (timestamp)
- `isUsed` (boolean) - True if used, False otherwise

- **Group**

Stores information about groups in our system. Attribute:

- `name` (string)

- **Permission**

Stores existing permissions of the system. Attributes:

- `name` (string)
- `description` (string)

- **GroupChange**

Stores information about updates made to groups (i.e., group logs). Attributes:

- `changeTime` (timestamp) - The date and time the group was updated.
- `changes` (JSONB) - A detailed log of the changes made to the group. The JSONB data type stores JSON data in a binary format, which improves performance by eliminating the need to reparse the data for each access.

- **Node**

Represents nodes or categories in the system. Attribute:

- `name` (string)

- **Chat**

Stores information about chats. Attribute:

- `name` (string)

- **Message**

Represents a message sent within a chat. Attributes:

- `text` (text) — The content of the message.
- `idempotencyKey` (UUID) — Used to ensure the message is not duplicated.
- `timestamp` (timestamp) — The date and time the message was sent.

- **MessageState**

Represents the different states a message can be in. Attributes:

- **name** (string)

- **MessageReport**

Represents reports submitted for messages. Attributes:

- **timestamp** (timestamp) — The date and time the message was reported.

9.1.2 Sensitive Database Entities

- **pseudonym_relations**

Enables ownership validation of pseudonyms through hashed ticket values.

- **pseudonymId** - Unique identifier of the pseudonym.
- **ticketHash** - Hashed version of the pseudonym ticket, used for verifying ownership.
- **ticketSalt** - Salt used during the hashing process.

- **admin_relations**

Stores encrypted identity mappings, used to reveal a user's identity if necessary.

- **pseudonymId** - Identifier of the pseudonym (from the Content DB).
- **encryptedUserId** - The user's id, encrypted with the System's public key.

- **pseudonym_wallets**

Used to persist pseudonym-ticket mappings that allow a user retrieve his pseudonyms.

- **userId** - Unique identifier of the user.
- **walletData** - JSON object containing the user pseudonyms.

- **pseudonym_creation_tickets**

Used to create pseudonyms for users. It is a one-time use ticket that is used to create a pseudonym.

- **userId** - Unique identifier of the ticket, together with chatId.
- **chatId** - Unique identifier of the ticket, together with userId.
- **pseudonymId** - Will be null when the ticket is punched.
- **timestamp**
- **punched** - True if the ticket has been used, False otherwise.

- **pseudonym_rate_limit**

Used to limit the number of pseudonyms a user can create in a given time period.

- **userId**

- chatId
- ticketId
- pseudonymId
- timestamp

9.2 API Endpoints

- **Home controller:** Exposes endpoints to retrieve the system status and to perform the initial configuration of the System:
 - GET /api/v1: Returns the status of the System
 - POST /api/v1/setup: Sets the initial configuration of the System, creating the root node, adding administrators and allowed domains.
- **Anonymity controller:** Exposes endpoints to enable pseudonym creation, confirmation and user identity unveiling.
 - POST /api/v1/anonymity/new: Creates a new pseudonym for the authenticated user in a specific chat, returning ticket to prove ownership of the pseudonym.
 - PUT /api/v1/anonymity/punch: Punches a ticket, confirming the ownership of the pseudonym.
 - POST /api/v1/anonymity/unveil: Unveils the identity of a user in a specific chat, revealing the user's real identity to the administrators.
- **User controller:** Exposes endpoints related to the User entity, providing detailed information about the user:
 - POST /api/v1/users/self: Returns the information of the authenticated user.
- **Authentication controller:** Exposes endpoints related to authentication and registration:
 - POST /api/v1/auth/register: Registers a new user in the System.
 - POST /api/v1/auth/verify-email: Verifies the email of a user.
 - POST /api/v1/auth/login: Authenticates a user and sets an Authorization cookie.
 - POST /api/v1/auth/logout: Logs out a user.
 - POST /api/v1/auth/forgot-password: Initiates the password reset process for a user.
 - POST /api/v1/auth/reset-password: Allows the user to reset his password.

- **Group controller:** Exposes endpoints related to the Group entity, which is responsible for managing the System's user's permissions:
 - POST /api/v1/groups: Creates a new group.
 - GET /api/v1/groups/{groupId}: Retrieves the details of a specific group.
 - PATCH /api/v1/groups/{groupId}: Updates the details of a specific group.
 - DELETE /api/v1/groups/{groupId}: Deletes a specific group.
- **Node controller:** Exposes endpoints related to the Node entity, which is responsible for customizing the System's tree structure:
 - POST /api/v1/nodes: Creates a new node in the System's tree structure.
 - GET /api/v1/nodes/{nodeId}: Retrieves the details of a specific node.
 - GET /api/v1/nodes/mine: Retrieves the nodes that the authenticated user has access to.
 - PATCH /api/v1/nodes/{nodeId}: Updates the details of a specific node.
 - DELETE /api/v1/nodes/{nodeId}: Deletes a specific node.
- **Chat controller:** Exposes endpoints related to the Chat entity, which is responsible for managing the System's chat rooms:
 - POST /api/categories/{categoryId}/chats: Creates a new chat room under a specific node.
 - GET /api/chats/{chatId}: Retrieves the details of a specific chat room, including the chat room's messages.
 - DELETE /api/chats/{chatId}/leave: Allows the authenticated user to leave a chat room.
 - GET /api/chats/{chatId}/favorite: Marks a specific chat room as favorite for the authenticated user.
 - DELETE /api/chats/{chatId}/favorite: Unmarks a specific chat room as favorite for the authenticated user.
 - GET /api/v1/chats/mine/favorites: Retrieves the list of favorite chat rooms for the authenticated user.
- **Message controller:** Exposes endpoints related to the Message entity, which is responsible for managing chat room messages in the System:
 - POST /api/v1/chats/{chatId}/messages: Sends a new message in a specific chat room.

- DELETE `/api/v1/chats/{chatId}/messages/{messageId}`: Deletes a specific message in a chat room.
- POST `/api/v1/chats/{chatId}/messages/report/{messageId}`: Reports a specific message in a chat room.
- PUT `/api/v1/chats/{chatId}/messages/report/{messageId}`: Clears the report of a specific message in a chat room.
- **SSE controller:** Exposes endpoints related to Server Sent Events (SSE), which are used to push real-time updates to the Frontend:
 - GET `/api/v1/sse/listen`: Subscribes to real-time updates for new messages in the chat rooms the user is participating in and for his favorite chat rooms.

Bibliography

- [1] Kristen P. Blair, Leslie C. Banes, and Lee Martin. Fostering noticing of classroom discussion features through analysis of contrasting cases. *Instructional Science*, 52(3):417–452, 2024.
- [2] Alan K. Goodboy Cathlin V. Clark-Gordon, Nicholas D. Bowman and Alyssa Wright. Anonymity and online self-disclosure: A meta-analysis. *Communication Reports*, 32(2):98–111, 2019.
- [3] Inc. DigitalOcean. Digitalocean: Cloud computing platform, 2025.
- [4] Inc. F5 Networks. Nginx documentation, 2025.
- [5] Oxen Privacy Tech Foundation. Session: Private messenger, 2025.
- [6] Signal Foundation. Signal: Speak freely, 2025.
- [7] The Matrix.org Foundation. Matrix: An open network for secure, decentralized communication, 2025.
- [8] Inc. GitHub. Github documentation, 2025.
- [9] Threema GmbH. Threema: The messenger that puts security and privacy first, 2025.
- [10] Google. Flutter documentation, 2025.
- [11] The PostgreSQL Global Development Group. Postgresql documentation, 2025.
- [12] M. Loureiro, N. Loureiro, and R. Silva. Differences of gender in oral and written communication apprehension of university students. *Education Sciences*, 2020.
- [13] Redis Ltd. Redis pub/sub documentation, 2025.
- [14] SimpleX Chat Ltd. Simplex chat: The first messaging network operating without user identifiers of any kind, 2025.
- [15] Robert Joseph McKee. Encouraging classroom discussion. *JSSE - Journal of Social Science Education*, 14(1):66–73, Oct. 2014.

- [16] P. K. Murphy, I. A. Wilkinson, A. O. Soter, M. N. Hennessey, and J. F. Alexander. Examining the effects of classroom discussion on students' comprehension of text: A meta-analysis. *Journal of Educational Psychology*, 101(3):740, 2009.
- [17] Lewis Nitschinsk, Stephanie Tobin, and Eric Vanman. The disinhibiting effects of anonymity increase online trolling. *Cyberpsychology, Behavior, and Social Networking*, 25, 05 2022.
- [18] Francisca Ossa. The role of anonymity in online harmful conducts: is regulation the answer? *Revista Chilena de Derecho y Tecnología*, 13, 04 2024.
- [19] Inc. Pivotal Software. Spring boot documentation, 2025.
- [20] Lynne D. Roberts and Camilla J Rajah-Kanagasabai. "i'd be so much more comfortable posting anonymously": Identified versus anonymous participation in student discussion boards. *Australasian Journal of Educational Technology*, 29(5), Nov. 2013.
- [21] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.
- [22] Jonathan Ying. The importance of the discussion method in the undergraduate business classroom. *Humanistic Management Journal*, 5(2):251–278, 2020.